

Trabajo Práctico N° 2:

Archivos Secuenciales Ordenados - Algorítmica Clásica.

Ejercicio 1.

Una empresa posee un archivo con información de los ingresos percibidos por diferentes empleados en concepto de comisión. De cada uno de ellos, se conoce: código de empleado, nombre y monto de la comisión. La información del archivo se encuentra ordenada por código de empleado y cada empleado puede aparecer más de una vez en el archivo de comisiones.

Realizar un procedimiento que reciba el archivo anteriormente descrito y lo compacte. En consecuencia, deberá generar un nuevo archivo en el cual cada empleado aparezca una única vez con el valor total de sus comisiones.

Nota: No se conoce, a priori, la cantidad de empleados. Además, el archivo debe ser recorrido una única vez.

```
program TP2_E1;
{$codepage UTF8}
uses crt, sysutils;
const
    codigo_salida=999;
type
    t_string20=string[20];
    t_registro_empleado=record
        codigo: int16;
        nombre: t_string20;
        comision: real;
    end;
    t_archivo_empleados=file of t_registro_empleado;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_empleados; var
archivo_carga_detalle: text);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_empleado do
            begin
                readln(archivo_carga_detalle,codigo,comision,nombre); nombre:=trim(nombre);
                write(archivo_detalle,registro_empleado);
            end;
        end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_empleado.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_empleado.nombre);
    textcolor(green); write('; Comisión: $'); textcolor(yellow);
    writeln(registro_empleado.comision:0:2);
end;
procedure imprimir_archivo_empleados(var archivo_empleados: t_archivo_empleados);
var
    registro_empleado: t_registro_empleado;
begin
```

```

reset(archivo_empleados);
while (not eof(archivo_empleados)) do
begin
    read(archivo_empleados,registro_empleado);
    imprimir_registro_empleado(registro_empleado);
end;
close(archivo_empleados);
end;
procedure leer_empleado(var archivo_detalle: t_archivo_empleados; var registro_empleado:
t_registro_empleado);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_empleado)
    else
        registro_empleado.codigo:=codigo_salida;
    end;
end;
procedure cargar_archivo_maestro(var archivo_maestro, archivo_detalle: t_archivo_empleados);
var
    registro_empleado_detalle, registro_empleado_maestro: t_registro_empleado;
    comision_total: real;
begin
    rewrite(archivo_maestro);
    reset(archivo_detalle);
    leer_empleado(archivo_detalle,registro_empleado_detalle);
    while (registro_empleado_detalle.codigo<>codigo_salida) do
    begin
        registro_empleado_maestro:=registro_empleado_detalle;
        comision_total:=0;
        while (registro_empleado_maestro.codigo=registro_empleado_detalle.codigo) do
        begin
            comision_total:=comision_total+registro_empleado_detalle.comision;
            leer_empleado(archivo_detalle,registro_empleado_detalle);
        end;
        registro_empleado_maestro.comision:=comision_total;
        write(archivo_maestro,registro_empleado_maestro);
    end;
    close(archivo_maestro);
    close(archivo_detalle);
end;
var
    archivo_detalle, archivo_maestro: t_archivo_empleados;
    archivo_carga_detalle: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
    assign(archivo_detalle,'E1_empleadosDetalle');
    assign(archivo_carga_detalle,'E1_empleadosDetalle.txt');
    cargar_archivo_detalle(archivo_detalle,archivo_carga_detalle);
    imprimir_archivo_empleados(archivo_detalle);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E1_empleadosMaestro');
    cargar_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_empleados(archivo_maestro);
end.

```

Ejercicio 2.

El encargado de ventas de un negocio de productos de limpieza desea administrar el stock de los productos que vende. Para ello, genera un archivo maestro donde figuran todos los productos que comercializa. De cada producto, se maneja la siguiente información: código de producto, nombre comercial, precio de venta, stock actual y stock mínimo. Diariamente, se genera un archivo detalle donde se registran todas las ventas de productos realizadas. De cada venta, se registran: código de producto y cantidad de unidades vendidas. Se pide realizar un programa con opciones para:

(a) Actualizar el archivo maestro con el archivo detalle, sabiendo que:

- Ambos archivos están ordenados por código de producto.
- Cada registro del maestro puede ser actualizado por 0, 1 o más registros del archivo detalle.
- El archivo detalle sólo contiene registros que están en el archivo maestro.

(b) Listar en un archivo de texto llamado "stock_minimo.txt" aquellos productos cuyo stock actual esté por debajo del stock mínimo permitido.

```
program TP2_E2;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=999;
  opcion_salida=0;
type
  t_string20=string[20];
  t_registro_producto=record
    codigo: int16;
    nombre: t_string20;
    precio: real;
    stock_actual: int16;
    stock_minimo: int16;
  end;
  t_registro_venta=record
    codigo: int16;
    cantidad_vendida: int16;
  end;
  t_archivo_maestro=file of t_registro_producto;
  t_archivo_detalle=file of t_registro_venta;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_producto: t_registro_producto;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_producto do
      begin
        readln(archivo_carga_maestro,codigo,precio,stock_actual,stock_minimo,nombre);
        nombre:=trim(nombre);
        write(archivo_maestro,registro_producto);
      end;
    end;
  close(archivo_maestro);
  close(archivo_carga_maestro);
  textcolor(green); writeln('El archivo binario maestro fue creado y cargado con éxito');
```

```
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,cantidad_vendida);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
    textcolor(green); writeln('El archivo binario detalle fue creado y cargado con éxito');
end;
procedure imprimir_registro_producto(registro_producto: t_registro_producto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_producto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_producto.nombre);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
write(registro_producto.precio:0:2);
    textcolor(green); write('; Stock actual: '); textcolor(yellow);
write(registro_producto.stock_actual);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
writeln(registro_producto.stock_minimo);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    reset(archivo_maestro);
    textcolor(green); writeln('Los productos del archivo maestro son: ');
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            imprimir_registro_producto(registro_producto);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Cantidad vendida: '); textcolor(yellow);
writeln(registro_venta.cantidad_vendida);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    textcolor(green); writeln('Las ventas del archivo detalle son: ');
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_venta)
```

```

else
    registro_venta.codigo:=codigo_salida;
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_producto: t_registro_producto;
    registro_venta: t_registro_venta;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    leer_venta(archivo_detalle,registro_venta);
    while (registro_venta.codigo<>codigo_salida) do
    begin
        read(archivo_maestro,registro_producto);
        while (registro_producto.codigo<>registro_venta.codigo) do
            read(archivo_maestro,registro_producto);
        while (registro_producto.codigo=registro_venta.codigo) do
        begin
            if (registro_venta.cantidad_vendida>=registro_producto.stock_actual) then
                registro_producto.stock_actual:=0
            else
                registro_producto.stock_actual:=registro_producto.stock_actual-
registro_venta.cantidad_vendida;
                leer_venta(archivo_detalle,registro_venta);
            end;
            seek(archivo_maestro,filepos(archivo_maestro)-1);
            write(archivo_maestro,registro_producto);
        end;
        close(archivo_maestro);
        close(archivo_detalle);
        textcolor(green); writeln('El archivo maestro fue actualizado con éxito');
    end;
procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
    archivo_txt: text;
begin
    reset(archivo_maestro);
    assign(archivo_txt,'E2_stock_minimo.txt'); rewrite(archivo_txt);
    textcolor(green); writeln('Los productos cuyo stock actual está por debajo del stock mínimo
son: ');
    while (not eof(archivo_maestro)) do
    begin
        read(archivo_maestro,registro_producto);
        if (registro_producto.stock_actual<registro_producto.stock_minimo) then
        begin
            imprimir_registro_producto(registro_producto);
            with registro_producto do
                writeln(archivo_txt,codigo,' ',nombre,' ',precio:0:2,' ',stock_actual,'
',stock_minimo);
            end;
        end;
        close(archivo_maestro);
        close(archivo_txt);
        textcolor(green); writeln('El archivo de texto "stock_minimo.txt" fue creado y cargado con
éxito');
    end;
procedure leer_opcion(var opcion: int8);
begin
    textcolor(red); writeln('MENÚ DE OPCIONES');
    textcolor(yellow); write('OPCIÓN 1: '); textcolor(green); writeln('Crear archivos de
registros ordenados de productos y cargarlos con datos ingresados desde archivos de texto');
    textcolor(yellow); write('OPCIÓN 2: '); textcolor(green); writeln('Listar en pantalla los
datos de los productos del archivo maestro');

```

```

    textcolor(yellow); write('OPCIÓN 3: '); textcolor(green); writeln('Listar en pantalla los
datos de los productos del archivo detalle');
    textcolor(yellow); write('OPCIÓN 4: '); textcolor(green); writeln('Actualizar el archivo
maestro con el archivo detalle');
    textcolor(yellow); write('OPCIÓN 5: '); textcolor(green); writeln('Listar en un archivo de
texto llamado "stock_minimo.txt" aquellos productos cuyo stock actual esté por debajo del
stock mínimo permitido');
    textcolor(yellow); write('OPCIÓN 0: '); textcolor(green); writeln('Salir del menú de
opciones');
    textcolor(green); write('Introducir opción elegida: '); textcolor(yellow); readln(opcion);
    writeln();
end;
procedure menu_opciones(var archivo_maestro: t_archivo_maestro; var archivo_detalle:
t_archivo_detalle; var archivo_carga_maestro, archivo_carga_detalle: text);
var
    opcion: int8;
begin
    leer_opcion(opcion);
    while (opcion<>opcion_salida) do
    begin
        case opcion of
            1:
                begin
                    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
                    cargar_archivo_detalle(archivo_detalle,archivo_carga_detalle);
                end;
            2: imprimir_archivo_maestro(archivo_maestro);
            3: imprimir_archivo_detalle(archivo_detalle);
            4: actualizar_archivo_maestro(archivo_maestro,archivo_detalle);
            5: exportar_archivo_txt(archivo_maestro);
            else
                textcolor(green); writeln('La opción ingresada no corresponde a ninguna del menú de
opciones');
            end;
            writeln();
            leer_opcion(opcion);
        end;
    end;
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle: t_archivo_detalle;
    archivo_carga_maestro, archivo_carga_detalle: text;
begin
    assign(archivo_maestro,'E2_productosMaestro');
    assign(archivo_carga_maestro,'E2_productosMaestro.txt');
    assign(archivo_detalle,'E2_ventasDetalle');
    assign(archivo_carga_detalle,'E2_ventasDetalle.txt');
    menu_opciones(archivo_maestro,archivo_detalle,archivo_carga_maestro,archivo_carga_detalle);
end.

```

Ejercicio 3.

A partir de información sobre la alfabetización en la Argentina, se necesita actualizar un archivo que contiene los siguientes datos: nombre de provincia, cantidad de personas alfabetizadas y total de encuestados. Se reciben dos archivos detalle provenientes de dos agencias de censo diferentes. Dichos archivos contienen: nombre de la provincia, código de localidad, cantidad de alfabetizados y cantidad de encuestados. Se pide realizar los módulos necesarios para actualizar el archivo maestro a partir de los dos archivos detalle.

Nota: Los archivos están ordenados por nombre de provincia y, en los archivos detalle, pueden venir 0, 1 o más registros por cada provincia.

```
program TP2_E3;
{$codepage UTF8}
uses crt, sysutils;
const
    nombre_salida='ZZZ';
type
    t_string50=string[50];
    t_registro_provincia1=record
        nombre: t_string50;
        alfabetizados: int16;
        encuestados: int16;
    end;
    t_registro_provincia2=record
        nombre: t_string50;
        codigo_localidad: int16;
        alfabetizados: int16;
        encuestados: int16;
    end;
    t_archivo_maestro=file of t_registro_provincia1;
    t_archivo_detalle=file of t_registro_provincia2;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_provincia: t_registro_provincia1;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_provincia do
            begin
                readln(archivo_carga_maestro,alfabetizados,encuestados,nombre); nombre:=trim(nombre);
                write(archivo_maestro,registro_provincia);
            end;
        end;
        close(archivo_maestro);
        close(archivo_carga_maestro);
    end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_provincia: t_registro_provincia2;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_provincia do
            begin
                readln(archivo_carga_detalle,codigo_localidad,alfabetizados,encuestados,nombre);
                nombre:=trim(nombre);
```

```
        write(archivo_detalle,registro_provincia);
    end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_provincia1(registro_provincia: t_registro_provincia1);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_provincia.nombre);
    textcolor(green); write('; Alfabetizados: '); textcolor(yellow);
write(registro_provincia.alfabetizados);
    textcolor(green); write('; Encuestados: '); textcolor(yellow);
writeln(registro_provincia.encuestados);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_provincia: t_registro_provincia1;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_provincia);
            imprimir_registro_provincia1(registro_provincia);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_provincia2(registro_provincia: t_registro_provincia2);
begin
    textcolor(green); write('Nombre: '); textcolor(yellow); write(registro_provincia.nombre);
    textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_provincia.codigo_localidad);
    textcolor(green); write('; Alfabetizados: '); textcolor(yellow);
write(registro_provincia.alfabetizados);
    textcolor(green); write('; Encuestados: '); textcolor(yellow);
writeln(registro_provincia.encuestados);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_provincia: t_registro_provincia2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_provincia);
            imprimir_registro_provincia2(registro_provincia);
        end;
    close(archivo_detalle);
end;
procedure leer_provincia(var archivo_detalle: t_archivo_detalle; var registro_provincia:
t_registro_provincia2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_provincia)
    else
        registro_provincia.nombre:=nombre_salida;
    end;
end;
procedure minimo(var archivo_detalle1, archivo_detalle2: t_archivo_detalle; var
registro_provincia1, registro_provincia2, min: t_registro_provincia2);
begin
    if (registro_provincia1.nombre<=registro_provincia2.nombre) then
        begin
            min:=registro_provincia1;
            leer_provincia(archivo_detalle1,registro_provincia1);
        end
    else
        begin
            min:=registro_provincia2;
```

```

    leer_provincia(archivo_detalle2, registro_provincia2);
end;
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle1, archivo_detalle2: t_archivo_detalle);
var
    registro_provincia: t_registro_provincia1;
    registro_provincia1, registro_provincia2, min: t_registro_provincia2;
begin
    reset(archivo_maestro);
    reset(archivo_detalle1); reset(archivo_detalle2);
    leer_provincia(archivo_detalle1, registro_provincia1);
    leer_provincia(archivo_detalle2, registro_provincia2);
    minimo(archivo_detalle1, archivo_detalle2, registro_provincia1, registro_provincia2, min);
    while (min.nombre <> nombre_salida) do
    begin
        read(archivo_maestro, registro_provincia);
        while (registro_provincia.nombre <> min.nombre) do
            read(archivo_maestro, registro_provincia);
        while (registro_provincia.nombre = min.nombre) do
        begin
            registro_provincia.alfabetizados := registro_provincia.alfabetizados + min.alfabetizados;
            registro_provincia.encuestados := registro_provincia.encuestados + min.encuestados;
            minimo(archivo_detalle1, archivo_detalle2, registro_provincia1, registro_provincia2, min);
        end;
        seek(archivo_maestro, filepos(archivo_maestro) - 1);
        write(archivo_maestro, registro_provincia);
    end;
    close(archivo_maestro);
    close(archivo_detalle1); close(archivo_detalle2);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle1, archivo_detalle2: t_archivo_detalle;
    archivo_carga_maestro, archivo_carga_detalle1, archivo_carga_detalle2: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro, 'E3_provinciasMaestro');
    assign(archivo_carga_maestro, 'E3_provinciasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 1:'); writeln();
    assign(archivo_detalle1, 'E3_provinciasDetalle1');
    assign(archivo_carga_detalle1, 'E3_provinciasDetalle1.txt');
    cargar_archivo_detalle(archivo_detalle1, archivo_carga_detalle1);
    imprimir_archivo_detalle(archivo_detalle1);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 2:'); writeln();
    assign(archivo_detalle2, 'E3_provinciasDetalle2');
    assign(archivo_carga_detalle2, 'E3_provinciasDetalle2.txt');
    cargar_archivo_detalle(archivo_detalle2, archivo_carga_detalle2);
    imprimir_archivo_detalle(archivo_detalle2);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro, archivo_detalle1, archivo_detalle2);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 4.

Se cuenta con un archivo de productos de una cadena de venta de alimentos congelados. De cada producto, se almacena: código del producto, nombre, descripción, stock disponible, stock mínimo y precio del producto.

Se recibe, diariamente, un archivo detalle de cada una de las 30 sucursales de la cadena. Se debe realizar el procedimiento que recibe los 30 detalles y actualiza el stock del archivo maestro. La información que se recibe en los detalles es: código de producto y cantidad vendida. Además, se deberá informar en un archivo de texto: nombre de producto, descripción, stock disponible y precio de aquellos productos que tengan stock disponible por debajo del stock mínimo. Pensar alternativas sobre realizar el informe en el mismo procedimiento de actualización o realizarlo en un procedimiento separado (analizar ventajas/desventajas en cada caso).

Nota: Todos los archivos se encuentran ordenados por código de producto. En cada detalle, puede venir 0 o N registros de un determinado producto.

```
program TP2_E4;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=30;
  codigo_salida=999;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_producto=record
    codigo: int16;
    nombre: t_string20;
    descripcion: t_string20;
    stock_disponible: int16;
    stock_minimo: int16;
    precio: real;
  end;
  t_registro_venta=record
    codigo: int16;
    cantidad_vendida: int16;
  end;
  t_archivo_maestro=file of t_registro_producto;
  t_archivo_detalle=file of t_registro_venta;
  t_vector_ventas=array[t_detalle] of t_registro_venta;
  t_vector_detalle=array[t_detalle] of t_archivo_detalle;
  t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_producto: t_registro_producto;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_producto do
      begin
        readln(archivo_carga_maestro,codigo,stock_disponible,stock_minimo,precio,nombre);
        nombre:=trim(nombre);
        readln(archivo_carga_maestro,descripcion);
        write(archivo_maestro,registro_producto);
      end;
    end;
  end;
```

```
close(archivo_maestro);
close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,cantidad_vendida);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_producto(registro_producto: t_registro_producto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_producto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_producto.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_producto.descripcion);
    textcolor(green); write('; Stock disponible: '); textcolor(yellow);
write(registro_producto.stock_disponible);
    textcolor(green); write('; Stock mínimo: '); textcolor(yellow);
write(registro_producto.stock_minimo);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
writeln(registro_producto.precio:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_producto: t_registro_producto;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_producto);
            imprimir_registro_producto(registro_producto);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Cantidad vendida: '); textcolor(yellow);
writeln(registro_venta.cantidad_vendida);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
```

```

    read(archivo_detalle,registro_venta)
  else
    registro_venta.codigo:=codigo_salida;
  end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);
var
  i, pos: t_detalle;
begin
  min.codigo:=codigo_salida;
  for i:= 1 to detalles_total do
    if (vector_ventas[i].codigo<min.codigo) then
      begin
        min:=vector_ventas[i];
        pos:=i;
      end;
    if (min.codigo<codigo_salida) then
      leer_venta(vector_detalle[pos],vector_ventas[pos]);
    end;
  end;
procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
  registro_producto: t_registro_producto;
  archivo_txt: text;
begin
  reset(archivo_maestro);
  assign(archivo_txt,'E4_stock_minimo.txt'); rewrite(archivo_txt);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_producto);
      if (registro_producto.stock_disponible<registro_producto.stock_minimo) then
        with registro_producto do
          writeln(archivo_txt,nombre,' ',descripcion,' ',stock_disponible,' ',precio:0:2);
        end;
      close(archivo_txt);
    end;
  end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
  registro_producto: t_registro_producto;
  min: t_registro_venta;
  vector_ventas: t_vector_ventas;
  i: t_detalle;
begin
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalle[i]);
      leer_venta(vector_detalle[i],vector_ventas[i]);
    end;
  minimo(vector_detalle,vector_ventas,min);
  while (min.codigo<>codigo_salida) do
    begin
      read(archivo_maestro,registro_producto);
      while (registro_producto.codigo<>min.codigo) do
        read(archivo_maestro,registro_producto);
      while (registro_producto.codigo=min.codigo) do
        begin
          if (min.cantidad_vendida>=registro_producto.stock_disponible) then
            registro_producto.stock_disponible:=0
          else
            registro_producto.stock_disponible:=registro_producto.stock_disponible-
min.cantidad_vendida;
            minimo(vector_detalle,vector_ventas,min);
          end;
        seek(archivo_maestro,filepos(archivo_maestro)-1);
        write(archivo_maestro,registro_producto);
      end;
    end;
  end;
end;

```

```
end;
exportar_archivo_txt(archivo_maestro);
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E4_productosMaestro');
    assign(archivo_carga_maestro,'E4_productosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i],'E4_ventasDetalle'+intToStr(i));
            assign(vector_carga_detalle[i],'E4_ventasDetalle'+intToStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 5.

Suponer que se trabaja en una oficina donde está montada una LAN (red local). La misma fue construida sobre una topología de red que conecta 5 máquinas entre sí y todas las máquinas se conectan con un servidor central. Semanalmente, cada máquina genera un archivo de logs informando las sesiones abiertas por cada usuario en cada terminal y por cuánto tiempo estuvo abierta. Cada archivo detalle contiene los siguientes campos: *cod_usuario*, *fecha*, *tiempo_sesion*. Se debe realizar un procedimiento que reciba los archivos detalle y genere un archivo maestro con los siguientes datos: *cod_usuario*, *fecha*, *tiempo_total_de_sesiones_abiertas*.

Notas:

- Cada archivo detalle está ordenado por *cod_usuario* y *fecha*.
- Un usuario puede iniciar más de una sesión el mismo día en la misma máquina o, inclusive, en diferentes máquinas.
- El archivo maestro debe crearse en la siguiente ubicación física: */var/log*.

```
program TP2_E5;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=3; // detalles_total=5;
    codigo_salida=999;
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_sesion=record
        codigo: int16;
        fecha: t_string20;
        tiempo: int16;
    end;
    t_archivo_sesiones=file of t_registro_sesion;
    t_vector_sesiones=array[t_detalle] of t_registro_sesion;
    t_vector_detalle=array[t_detalle] of t_archivo_sesiones;
    t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_sesiones; var
archivo_carga_detalle: text);
var
    registro_sesion: t_registro_sesion;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_sesion do
            begin
                readln(archivo_carga_detalle,codigo,tiempo,fecha); fecha:=trim(fecha);
                write(archivo_detalle,registro_sesion);
            end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_sesion(registro_sesion: t_registro_sesion);
begin
    textcolor(green); write('Código de usuario: '); textcolor(yellow);
    write(registro_sesion.codigo);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_sesion.fecha);
    textcolor(green); write('; Tiempo: '); textcolor(yellow); writeln(registro_sesion.tiempo);
end;
```

```
procedure imprimir_archivo_sesiones(var archivo_sesiones: t_archivo_sesiones);
var
    registro_sesion: t_registro_sesion;
begin
    reset(archivo_sesiones);
    while (not eof(archivo_sesiones)) do
        begin
            read(archivo_sesiones, registro_sesion);
            imprimir_registro_sesion(registro_sesion);
        end;
    close(archivo_sesiones);
end;

procedure leer_sesion(var archivo_detalle: t_archivo_sesiones; var registro_sesion:
t_registro_sesion);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_sesion)
    else
        registro_sesion.codigo:=codigo_salida;
    end;

procedure minimo(var vector_detalle: t_vector_detalle; var vector_sesiones:
t_vector_sesiones; var min: t_registro_sesion);
var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida;
    for i:= 1 to detalles_total do
        if ((vector_sesiones[i].codigo<min.codigo) or ((vector_sesiones[i].codigo=min.codigo) and
(vector_sesiones[i].fecha<min.fecha))) then
            begin
                min:=vector_sesiones[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida) then
            leer_sesion(vector_detalle[pos], vector_sesiones[pos]);
        end;
    end;

procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_sesiones; var vector_detalle:
t_vector_detalle);
var
    registro_sesion, min: t_registro_sesion;
    vector_sesiones: t_vector_sesiones;
    i: t_detalle;
begin
    rewrite(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_sesion(vector_detalle[i], vector_sesiones[i]);
        end;
    minimo(vector_detalle, vector_sesiones, min);
    while (min.codigo<>codigo_salida) do
        begin
            registro_sesion.codigo:=min.codigo;
            while (registro_sesion.codigo=min.codigo) do
                begin
                    registro_sesion.fecha:=min.fecha;
                    registro_sesion.tiempo:=0;
                    while ((registro_sesion.codigo=min.codigo) and (registro_sesion.fecha=min.fecha)) do
                        begin
                            registro_sesion.tiempo:=registro_sesion.tiempo+min.tiempo;
                            minimo(vector_detalle, vector_sesiones, min);
                        end;
                    write(archivo_maestro, registro_sesion);
                end;
            end;
        end;
    close(archivo_maestro);
```

```
    for i:= 1 to detalles_total do
        close(vector_detalle[i]);
    end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_sesiones;
    i: t_detalle;
begin
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i], 'E5_sesionesDetalle'+inttoStr(i));
            assign(vector_carga_detalle[i], 'E5_sesionesDetalle'+inttoStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i], vector_carga_detalle[i]);
            imprimir_archivo_sesiones(vector_detalle[i]);
        end;
        writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
        assign(archivo_maestro, 'E5_sesionesMaestro');
        cargar_archivo_maestro(archivo_maestro, vector_detalle);
        imprimir_archivo_sesiones(archivo_maestro);
    end.
end.
```

Ejercicio 6.

Se desea modelar la información necesaria para un sistema de recuentos de casos de COVID para el Ministerio de Salud de la Provincia de Buenos Aires.

Diariamente, se reciben archivos provenientes de los distintos municipios. La información contenida en los mismos es la siguiente: código de localidad, código cepa, cantidad de casos activos, cantidad de casos nuevos, cantidad de casos recuperados, cantidad de casos fallecidos.

El ministerio cuenta con un archivo maestro con la siguiente información: código localidad, nombre localidad, código cepa, nombre cepa, cantidad de casos activos, cantidad de casos nuevos, cantidad de recuperados y cantidad de fallecidos.

Se debe realizar el procedimiento que permita actualizar el maestro con los detalles recibidos, se reciben 10 detalles. Todos los archivos están ordenados por código de localidad y código de cepa.

Para la actualización, se debe proceder de la siguiente manera:

- *Al número de fallecidos se le suman el valor de fallecidos recibido del detalle.*
- *Ídem anterior para los recuperados.*
- *Los casos activos se actualizan con el valor recibido en el detalle.*
- *Ídem anterior para los casos nuevos hallados.*

Realizar las declaraciones necesarias, el programa principal y los procedimientos que requiera para la actualización solicitada e informar cantidad de localidades con más de 50 casos activos (las localidades pueden o no haber sido actualizadas).

```

program TP2_E6;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=10;
  codigo_salida_detalle=999;
  codigo_salida_maestro=9999;
  casos_activos_corte=50;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_localidad1=record
    codigo: int16;
    nombre: t_string20;
    codigo_cepa: int16;
    nombre_cepa: t_string20;
    casos_activos: int16;
    casos_nuevos: int16;
    casos_recuperados: int16;
    casos_fallecidos: int16;
  end;
  t_registro_localidad2=record
    codigo: int16;
    codigo_cepa: int16;
    casos_activos: int32;
    casos_nuevos: int16;

```

```

    casos_recuperados: int16;
    casos_fallecidos: int16;
end;
t_archivo_maestro=file of t_registro_localidad1;
t_archivo_detalle=file of t_registro_localidad2;
t_vector_localidades=array[t_detalle] of t_registro_localidad2;
t_vector_detalle=array[t_detalle] of t_archivo_detalle;
t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_localidad: t_registro_localidad1;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_localidad do
            begin
                readln(archivo_carga_maestro,codigo,codigo_cepa,casos_activos,casos_nuevos,casos_recuper
ados,casos_fallecidos,nombre_cepa); nombre_cepa:=trim(nombre_cepa);
                readln(archivo_carga_maestro,nombre);
                write(archivo_maestro,registro_localidad);
            end;
        end;
        close(archivo_maestro);
        close(archivo_carga_maestro);
    end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_localidad: t_registro_localidad2;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_localidad do
            begin
                readln(archivo_carga_detalle,codigo,codigo_cepa,casos_activos,casos_nuevos,casos_recuper
ados,casos_fallecidos);
                write(archivo_detalle,registro_localidad);
            end;
        end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_localidad1(registro_localidad: t_registro_localidad1);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_localidad.codigo);
    textcolor(green); write('; Nombre de localidad: '); textcolor(yellow);
write(registro_localidad.nombre);
    textcolor(green); write('; Código de cepa: '); textcolor(yellow);
write(registro_localidad.codigo_cepa);
    textcolor(green); write('; Nombre de cepa: '); textcolor(yellow);
write(registro_localidad.nombre_cepa);
    textcolor(green); write('; Casos activos: '); textcolor(yellow);
write(registro_localidad.casos_activos);
    textcolor(green); write('; Casos nuevos: '); textcolor(yellow);
write(registro_localidad.casos_nuevos);
    textcolor(green); write('; Casos recuperados: '); textcolor(yellow);
write(registro_localidad.casos_recuperados);
    textcolor(green); write('; Casos fallecidos: '); textcolor(yellow);
writeln(registro_localidad.casos_fallecidos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_localidad: t_registro_localidad1;
begin

```

```

reset(archivo_maestro);
while (not eof(archivo_maestro)) do
begin
    read(archivo_maestro,registro_localidad);
    imprimir_registro_localidad1(registro_localidad);
end;
close(archivo_maestro);
end;
procedure imprimir_registro_localidad2(registro_localidad: t_registro_localidad2);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_localidad.codigo);
    textcolor(green); write('; Código de cepa: '); textcolor(yellow);
write(registro_localidad.codigo_cepa);
    textcolor(green); write('; Casos activos: '); textcolor(yellow);
write(registro_localidad.casos_activos);
    textcolor(green); write('; Casos nuevos: '); textcolor(yellow);
write(registro_localidad.casos_nuevos);
    textcolor(green); write('; Casos recuperados: '); textcolor(yellow);
write(registro_localidad.casos_recuperados);
    textcolor(green); write('; Casos fallecidos: '); textcolor(yellow);
writeln(registro_localidad.casos_fallecidos);
end;
procedure imprimir_archivo_detalles(var archivo_detalle: t_archivo_detalle);
var
    registro_localidad: t_registro_localidad2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
    begin
        read(archivo_detalle,registro_localidad);
        imprimir_registro_localidad2(registro_localidad);
    end;
    close(archivo_detalle);
end;
procedure leer_localidad_detalle(var archivo_detalle: t_archivo_detalle; var
registro_localidad: t_registro_localidad2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_localidad)
    else
        registro_localidad.codigo:=codigo_salida_detalle;
    end;
procedure minimo(var vector_detalles: t_vector_detalles; var vector_localidades:
t_vector_localidades; var min: t_registro_localidad2);
var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida_detalle;
    for i:= 1 to detalles_total do
        if ((vector_localidades[i].codigo<min.codigo) or
((vector_localidades[i].codigo=min.codigo) and
(vector_localidades[i].codigo_cepa<min.codigo_cepa))) then
            begin
                min:=vector_localidades[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida_detalle) then
            leer_localidad_detalle(vector_detalles[pos],vector_localidades[pos]);
        end;
procedure actualizar_localidades_corte(casos_activos_localidad: int32; var localidades_corte:
int16);
begin
    if (casos_activos_localidad>casos_activos_corte) then
        localidades_corte:=localidades_corte+1;
end;

```

```

procedure leer_localidad_maestro(var archivo_maestro: t_archivo_maestro; var
registro_localidad: t_registro_localidad1);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_localidad)
    else
        registro_localidad.codigo:=codigo_salida_maestro;
    end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_localidad, registro_localidad_actual: t_registro_localidad1;
    min: t_registro_localidad2;
    vector_localidades: t_vector_localidades;
    i: t_detalle;
    localidades_corte: int16;
    casos_activos_localidad: int32;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_localidad_detalle(vector_detalle[i],vector_localidades[i]);
        end;
    minimo(vector_detalle,vector_localidades,min);
    while (min.codigo<>codigo_salida_detalle) do
        begin
            casos_activos_localidad:=0;
            leer_localidad_maestro(archivo_maestro,registro_localidad);
            while (registro_localidad.codigo<>min.codigo) do
                begin
                    registro_localidad_actual:=registro_localidad;
                    while (registro_localidad.codigo=registro_localidad_actual.codigo) do
                        begin
                            casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
                            leer_localidad_maestro(archivo_maestro,registro_localidad);
                        end;
                    actualizar_localidades_corte(casos_activos_localidad,localidades_corte);
                    casos_activos_localidad:=0;
                end;
            ok:=true;
            registro_localidad_actual:=registro_localidad;
            while (registro_localidad_actual.codigo=min.codigo) do
                begin
                    while (registro_localidad.codigo_cepa<>min.codigo_cepa) do
                        begin
                            if (ok=true) then
                                casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
                                read(archivo_maestro,registro_localidad);
                                ok:=true;
                            end;
                            while ((registro_localidad.codigo=min.codigo) and
(registro_localidad.codigo_cepa=min.codigo_cepa)) do
                                begin
                                    registro_localidad.casos_fallecidos:=registro_localidad.casos_fallecidos+min.casos_fal
lecidos;
                                    registro_localidad.casos_recuperados:=registro_localidad.casos_recuperados+min.casos_r
ecuperados;
                                    registro_localidad.casos_activos:=min.casos_activos;
                                    registro_localidad.casos_nuevos:=min.casos_nuevos;
                                    minimo(vector_detalle,vector_localidades,min);
                                end;
                                seek(archivo_maestro,filepos(archivo_maestro)-1);
                                write(archivo_maestro,registro_localidad);

```

```

    casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
    ok:=false;
    if (registro_localidad_actual.codigo<>min.codigo) then
    begin
        leer_localidad_maestro(archivo_maestro,registro_localidad);
        while (registro_localidad.codigo=registro_localidad_actual.codigo) do
        begin
            casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
            leer_localidad_maestro(archivo_maestro,registro_localidad);
        end;
        seek(archivo_maestro,filepos(archivo_maestro)-1);
    end;
    end;
    actualizar_localidades_corte(casos_activos_localidad,localidades_corte);
end;
casos_activos_localidad:=0;
leer_localidad_maestro(archivo_maestro,registro_localidad);
while (registro_localidad.codigo<>codigo_salida_maestro) do
begin
    registro_localidad_actual.codigo:=registro_localidad.codigo;
    while (registro_localidad.codigo=registro_localidad_actual.codigo) do
    begin
        casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
        leer_localidad_maestro(archivo_maestro,registro_localidad);
    end;
    actualizar_localidades_corte(casos_activos_localidad,localidades_corte);
    casos_activos_localidad:=0;
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con más de '); textcolor(yellow);
write(casos_activos_corte); textcolor(green); write(' casos activos es '); textcolor(red);
writeln(localidades_corte);
    writeln();
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_localidad: t_registro_localidad1;
    min: t_registro_localidad2;
    vector_localidades: t_vector_localidades;
    i: t_detalle;
    localidad, localidades_corte: int16;
    casos_activos_localidad: int32;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
    begin
        reset(vector_detalle[i]);
        leer_localidad_detalle(vector_detalle[i],vector_localidades[i]);
    end;
    minimo(vector_detalle,vector_localidades,min);
    leer_localidad_maestro(archivo_maestro,registro_localidad);
    while (registro_localidad.codigo<>codigo_salida_maestro) do
    begin
        localidad:=registro_localidad.codigo;
        casos_activos_localidad:=0;
        while (registro_localidad.codigo=localidad) do
        begin
            ok:=false;
            while ((registro_localidad.codigo=min.codigo) and
(registro_localidad.codigo_cepa=min.codigo_cepa)) do
                begin

```

```

        ok:=true;
        registro_localidad.casos_fallecidos:=registro_localidad.casos_fallecidos+min.casos_fal
lecidos;
        registro_localidad.casos_recuperados:=registro_localidad.casos_recuperados+min.casos_r
ecuperados;
        registro_localidad.casos_activos:=min.casos_activos;
        registro_localidad.casos_nuevos:=min.casos_nuevos;
        minimo(vector_detalle, vector_localidades, min);
    end;
    if (ok=true) then
    begin
        seek(archivo_maestro, filepos(archivo_maestro)-1);
        write(archivo_maestro, registro_localidad);
    end;
    casos_activos_localidad:=casos_activos_localidad+registro_localidad.casos_activos;
    leer_localidad_maestro(archivo_maestro, registro_localidad);
    end;
    actualizar_localidades_corte(casos_activos_localidad, localidades_corte);
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con más de '); textcolor(yellow);
write(casos_activos_corte); textcolor(green); write(' casos activos es '); textcolor(red);
writeln(localidades_corte);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro, 'E6_localidadesMaestro');
    assign(archivo_carga_maestro, 'E6_localidadesMaestro.txt');
    cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
    begin
        writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ', i, ':'); writeln();
        assign(vector_detalle[i], 'E6_localidadesDetalle'+inttoStr(i));
        assign(vector_carga_detalle[i], 'E6_localidadesDetalle'+inttoStr(i)+'.txt');
        cargar_archivo_detalle(vector_detalle[i], vector_carga_detalle[i]);
        imprimir_archivo_detalle(vector_detalle[i]);
    end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    //actualizar1_archivo_maestro(archivo_maestro, vector_detalle);
    actualizar2_archivo_maestro(archivo_maestro, vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 7.

Se dispone de un archivo maestro con información de los alumnos de la Facultad de Informática. Cada registro del archivo maestro contiene: código de alumno, apellido, nombre, cantidad de cursadas aprobadas y cantidad de materias con final aprobado. El archivo maestro está ordenado por código de alumno.

Además, se tienen dos archivos detalle con información sobre el desempeño académico de los alumnos: un archivo de cursadas y un archivo de exámenes finales. El archivo de cursadas contiene información sobre las materias cursadas por los alumnos. Cada registro incluye: código de alumno, código de materia, año de cursada y resultado (sólo interesa si la cursada fue aprobada o desaprobada). Por su parte, el archivo de exámenes finales contiene información sobre los exámenes finales rendidos. Cada registro incluye: código de alumno, código de materia, fecha del examen y nota obtenida. Ambos archivos detalle están ordenados por código de alumno y código de materia, y pueden contener 0, 1 o más registros por alumno en el archivo maestro. Un alumno podría cursar una materia muchas veces, así como también podría rendir el final de una materia en múltiples ocasiones.

Se debe desarrollar un programa que actualice el archivo maestro, ajustando la cantidad de cursadas aprobadas y la cantidad de materias con final aprobado, utilizando la información de los archivos detalle. Las reglas de actualización son las siguientes:

- *Si un alumno aprueba una cursada, se incrementa en uno la cantidad de cursadas aprobadas.*
- *Si un alumno aprueba un examen final (nota ≥ 4), se incrementa en uno la cantidad de materias con final aprobado.*

Notas:

- *Los archivos deben procesarse en un único recorrido.*
- *No es necesario comprobar que no haya inconsistencias en la información de los archivos detalles. Esto es, no puede suceder que un alumno apruebe más de una vez la cursada de una misma materia (a lo sumo, la aprueba una vez), algo similar ocurre con los exámenes finales.*

```
program TP2_E7;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=999;
  resultado_corte='Aprobada'; nota_corte=4.0;
  anio_ini=2000; anio_fin=2025;
type
  t_string20=string[20];
  t_anio=anio_ini..anio_fin;
  t_registro_alumno=record
    codigo: int16;
    apellido: t_string20;
    nombre: t_string20;
    cursadas_aprobadas: int16;
    finales_aprobados: int16;
  end;
```

```

t_registro_cursada=record
  codigo: int16;
  codigo_materia: int16;
  anio: t_anio;
  resultado: t_string20;
end;
t_registro_final=record
  codigo: int16;
  codigo_materia: int16;
  fecha: t_string20;
  nota: real;
end;
t_registro_cursada_final=record
  codigo: int16;
  codigo_materia: int16;
  resultado: t_string20;
  nota: real;
end;
t_archivo_maestro=file of t_registro_alumno;
t_archivo_detalle1=file of t_registro_cursada;
t_archivo_detalle2=file of t_registro_final;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_alumno: t_registro_alumno;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_alumno do
      begin
        readln(archivo_carga_maestro,codigo,cursadas_aprobadas, finales_aprobados,apellido);
        apellido:=trim(apellido);
        readln(archivo_carga_maestro,nombre);
        write(archivo_maestro,registro_alumno);
      end;
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
  end;
procedure cargar_archivo_detalle1(var archivo_detalle1: t_archivo_detalle1; var
archivo_carga_detalle1: text);
var
  registro_cursada: t_registro_cursada;
begin
  rewrite(archivo_detalle1);
  reset(archivo_carga_detalle1);
  while (not eof(archivo_carga_detalle1)) do
    with registro_cursada do
      begin
        readln(archivo_carga_detalle1,codigo,codigo_materia,anio,resultado);
        resultado:=trim(resultado);
        write(archivo_detalle1,registro_cursada);
      end;
    end;
    close(archivo_detalle1);
    close(archivo_carga_detalle1);
  end;
procedure cargar_archivo_detalle2(var archivo_detalle2: t_archivo_detalle2; var
archivo_carga_detalle2: text);
var
  registro_final: t_registro_final;
begin
  rewrite(archivo_detalle2);
  reset(archivo_carga_detalle2);
  while (not eof(archivo_carga_detalle2)) do
    with registro_final do
      begin

```

```

        readln(archivo_carga_detalle2,codigo,codigo_materia,nota,fecha); fecha:=trim(fecha);
        write(archivo_detalle2,registro_final);
    end;
    close(archivo_detalle2);
    close(archivo_carga_detalle2);
end;
procedure imprimir_registro_alumno(registro_alumno: t_registro_alumno);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_alumno.codigo);
    textcolor(green); write('; Apellido: '); textcolor(yellow); write(registro_alumno.apellido);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_alumno.nombre);
    textcolor(green); write('; Cursadas aprobadas: '); textcolor(yellow);
write(registro_alumno.cursadas_aprobadas);
    textcolor(green); write('; Finales aprobados: '); textcolor(yellow);
writeln(registro_alumno.finales_aprobados);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_alumno: t_registro_alumno;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_alumno);
            imprimir_registro_alumno(registro_alumno);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_cursada(registro_cursada: t_registro_cursada);
begin
    textcolor(green); write('Código de alumno: '); textcolor(yellow);
write(registro_cursada.codigo);
    textcolor(green); write('; Código de materia: '); textcolor(yellow);
write(registro_cursada.codigo_materia);
    textcolor(green); write('; Año: '); textcolor(yellow); write(registro_cursada.anio);
    textcolor(green); write('; Resultado: '); textcolor(yellow);
writeln(registro_cursada.resultado);
end;
procedure imprimir_archivo_detalle1(var archivo_detalle1: t_archivo_detalle1);
var
    registro_cursada: t_registro_cursada;
begin
    reset(archivo_detalle1);
    while (not eof(archivo_detalle1)) do
        begin
            read(archivo_detalle1,registro_cursada);
            imprimir_registro_cursada(registro_cursada);
        end;
    close(archivo_detalle1);
end;
procedure imprimir_registro_final(registro_final: t_registro_final);
begin
    textcolor(green); write('Código de alumno: '); textcolor(yellow);
write(registro_final.codigo);
    textcolor(green); write('; Código de materia: '); textcolor(yellow);
write(registro_final.codigo_materia);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_final.fecha);
    textcolor(green); write('; Nota: '); textcolor(yellow); writeln(registro_final.nota:0:2);
end;
procedure imprimir_archivo_detalle2(var archivo_detalle2: t_archivo_detalle2);
var
    registro_final: t_registro_final;
begin
    reset(archivo_detalle2);
    while (not eof(archivo_detalle2)) do
        begin

```

```

    read(archivo_detalle2,registro_final);
    imprimir_registro_final(registro_final);
end;
close(archivo_detalle2);
end;
procedure leer_cursada(var archivo_detalle1: t_archivo_detalle1; var registro_cursada:
t_registro_cursada);
begin
    if (not eof(archivo_detalle1)) then
        read(archivo_detalle1,registro_cursada)
    else
        registro_cursada.codigo:=codigo_salida;
end;
procedure leer_final(var archivo_detalle2: t_archivo_detalle2; var registro_final:
t_registro_final);
begin
    if (not eof(archivo_detalle2)) then
        read(archivo_detalle2,registro_final)
    else
        registro_final.codigo:=codigo_salida;
end;
procedure minimo(var archivo_detalle1: t_archivo_detalle1; var archivo_detalle2:
t_archivo_detalle2; var registro_cursada: t_registro_cursada; var registro_final:
t_registro_final; var min: t_registro_cursada_final);
begin
    if (registro_cursada.codigo<=registro_final.codigo) then
        begin
            min.codigo:=registro_cursada.codigo;
            min.codigo_materia:=registro_cursada.codigo_materia;
            min.resultado:=registro_cursada.resultado;
            min.nota:=0;
            leer_cursada(archivo_detalle1,registro_cursada);
        end
    else
        begin
            min.codigo:=registro_final.codigo;
            min.codigo_materia:=registro_final.codigo_materia;
            min.resultado:='';
            min.nota:=registro_final.nota;
            leer_final(archivo_detalle2,registro_final);
        end;
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle1: t_archivo_detalle1; var archivo_detalle2: t_archivo_detalle2);
var
    registro_alumno: t_registro_alumno;
    registro_cursada: t_registro_cursada;
    registro_final: t_registro_final;
    min: t_registro_cursada_final;
begin
    reset(archivo_maestro);
    reset(archivo_detalle1); reset(archivo_detalle2);
    leer_cursada(archivo_detalle1,registro_cursada);
    leer_final(archivo_detalle2,registro_final);
    minimo(archivo_detalle1,archivo_detalle2,registro_cursada,registro_final,min);
    while (min.codigo<>codigo_salida) do
        begin
            read(archivo_maestro,registro_alumno);
            while (registro_alumno.codigo<>min.codigo) do
                read(archivo_maestro,registro_alumno);
            while (registro_alumno.codigo=min.codigo) do
                begin
                    if (min.resultado=resultado_corte) then
                        registro_alumno.cursadas_aprobadas:=registro_alumno.cursadas_aprobadas+1;
                    if (min.nota>nota_corte) then
                        registro_alumno.finales_aprobados:=registro_alumno.finales_aprobados+1;

```

```
        minimo(archivo_detalle1,archivo_detalle2,registro_cursada,registro_final,min);
    end;
    seek(archivo_maestro,filepos(archivo_maestro)-1);
    write(archivo_maestro,registro_alumno);
end;
close(archivo_maestro);
close(archivo_detalle1); close(archivo_detalle2);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle1: t_archivo_detalle1;
    archivo_detalle2: t_archivo_detalle2;
    archivo_carga_maestro, archivo_carga_detalle1, archivo_carga_detalle2: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E7_alumnosMaestro');
assign(archivo_carga_maestro,'E7_alumnosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 1:'); writeln();
    assign(archivo_detalle1,'E7_cursadasDetalle');
assign(archivo_carga_detalle1,'E7_cursadasDetalle.txt');
    cargar_archivo_detalle1(archivo_detalle1,archivo_carga_detalle1);
    imprimir_archivo_detalle1(archivo_detalle1);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE 2:'); writeln();
    assign(archivo_detalle2,'E7_finalesDetalle');
assign(archivo_carga_detalle2,'E7_finalesDetalle.txt');
    cargar_archivo_detalle2(archivo_detalle2,archivo_carga_detalle2);
    imprimir_archivo_detalle2(archivo_detalle2);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,archivo_detalle1,archivo_detalle2);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 8.

Se quiere optimizar la gestión del consumo de yerba mate en distintas provincias de Argentina. Para ello, se cuenta con un archivo maestro que contiene la siguiente información: código de provincia, nombre de la provincia, cantidad de habitantes y cantidad total de kilos de yerba consumidos históricamente.

Cada mes, se reciben 16 archivos de relevamiento con información sobre el consumo de yerba en los distintos puntos del país. Cada archivo contiene: código de provincia y cantidad de kilos de yerba consumidos en ese relevamiento. Un archivo de relevamiento puede contener información de una o varias provincias, y una misma provincia puede aparecer cero, una o más veces en distintos archivos de relevamiento.

Tanto el archivo maestro como los archivos de relevamiento están ordenados por código de provincia.

Se desea realizar un programa que actualice el archivo maestro en base a la nueva información de consumo de yerba. Además, se debe informar en pantalla aquellas provincias (código y nombre) donde la cantidad total de yerba consumida supere los 10.000 kilos históricamente, junto con el promedio consumido de yerba por habitante. Es importante tener en cuenta tanto las provincias actualizadas como las que no fueron actualizadas.

Nota: Cada archivo debe recorrerse una única vez.

```
program TP2_E8;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=16;
  codigo_salida=999;
  kilos_corte=10000;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_provincia1=record
    codigo: int16;
    nombre: t_string20;
    habitantes: int16;
    kilos: int32;
  end;
  t_registro_provincia2=record
    codigo: int16;
    kilos: int32;
  end;
  t_archivo_maestro=file of t_registro_provincia1;
  t_archivo_detalle=file of t_registro_provincia2;
  t_vector_provincias=array[t_detalle] of t_registro_provincia2;
  t_vector_detalle=array[t_detalle] of t_archivo_detalle;
  t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_provincia: t_registro_provincia1;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
```

```
while (not eof(archivo_carga_maestro)) do
  with registro_provincia do
    begin
      readln(archivo_carga_maestro,codigo,habitantes,kilos,nombre); nombre:=trim(nombre);
      write(archivo_maestro,registro_provincia);
    end;
  close(archivo_maestro);
  close(archivo_carga_maestro);
end;

procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
  registro_provincia: t_registro_provincia2;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_provincia do
      begin
        readln(archivo_carga_detalle,codigo,kilos);
        write(archivo_detalle,registro_provincia);
      end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
  end;
end;

procedure imprimir_registro_provincia1(registro_provincia: t_registro_provincia1);
begin
  textcolor(green); write('Código: '); textcolor(yellow); write(registro_provincia.codigo);
  textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_provincia.nombre);
  textcolor(green); write('; Habitantes: '); textcolor(yellow);
write(registro_provincia.habitantes);
  textcolor(green); write('; Kilos de yerba consumidos: '); textcolor(yellow);
writeln(registro_provincia.kilos);
end;

procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_provincia: t_registro_provincia1;
begin
  reset(archivo_maestro);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_provincia);
      imprimir_registro_provincia1(registro_provincia);
    end;
  close(archivo_maestro);
end;

procedure imprimir_registro_provincia2(registro_provincia: t_registro_provincia2);
begin
  textcolor(green); write('Código: '); textcolor(yellow); write(registro_provincia.codigo);
  textcolor(green); write('; Kilos de yerba consumidos: '); textcolor(yellow);
writeln(registro_provincia.kilos);
end;

procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
  registro_provincia: t_registro_provincia2;
begin
  reset(archivo_detalle);
  while (not eof(archivo_detalle)) do
    begin
      read(archivo_detalle,registro_provincia);
      imprimir_registro_provincia2(registro_provincia);
    end;
  close(archivo_detalle);
end;

procedure leer_provincia_detalle(var archivo_detalle: t_archivo_detalle; var
registro_provincia: t_registro_provincia2);
```

```

begin
  if (not eof(archivo_detalle)) then
    read(archivo_detalle, registro_provincia)
  else
    registro_provincia.codigo:=codigo_salida;
end;
procedure minimo(var vector_detalles: t_vector_detalles; var vector_provincias:
t_vector_provincias; var min: t_registro_provincia2);
var
  i, pos: t_detalle;
begin
  min.codigo:=codigo_salida;
  for i:= 1 to detalles_total do
    if (vector_provincias[i].codigo<min.codigo) then
      begin
        min:=vector_provincias[i];
        pos:=i;
      end;
    if (min.codigo<codigo_salida) then
      leer_provincia_detalle(vector_detalles[pos], vector_provincias[pos]);
end;
procedure imprimir_texto(registro_provincia: t_registro_provincia1);
begin
  if (registro_provincia.kilos>kilos_corte) then
    begin
      textcolor(green); write('El nombre y el código de esta provincia donde la cantidad total
de yerba consumida supera los '); textcolor(yellow); write(kilos_corte); textcolor(green);
write(' kilos son '); textcolor(red); write(registro_provincia.nombre); textcolor(green);
write(' y '); textcolor(red); write(registro_provincia.codigo); textcolor(green); write(',
respectivamente, mientras que el promedio consumido de yerba por habitante es ');
textcolor(red); writeln(registro_provincia.kilos/registro_provincia.habitantes:0:2);
    end;
end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalles: t_vector_detalles);
var
  registro_provincia: t_registro_provincia1;
  min: t_registro_provincia2;
  vector_provincias: t_vector_provincias;
  i: t_detalle;
begin
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalles[i]);
      leer_provincia_detalle(vector_detalles[i], vector_provincias[i]);
    end;
  minimo(vector_detalles, vector_provincias, min);
  while (min.codigo<>codigo_salida) do
    begin
      read(archivo_maestro, registro_provincia);
      while (registro_provincia.codigo<>min.codigo) do
        begin
          imprimir_texto(registro_provincia);
          read(archivo_maestro, registro_provincia);
        end;
      while (registro_provincia.codigo=min.codigo) do
        begin
          registro_provincia.kilos:=registro_provincia.kilos+min.kilos;
          minimo(vector_detalles, vector_provincias, min);
        end;
      seek(archivo_maestro, filepos(archivo_maestro)-1);
      write(archivo_maestro, registro_provincia);
      imprimir_texto(registro_provincia);
    end;
  while (not eof(archivo_maestro)) do

```

```

begin
    read(archivo_maestro,registro_provincia);
    imprimir_texto(registro_provincia);
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
writeln();
end;
procedure leer_provincia_maestro(var archivo_maestro: t_archivo_maestro; var
registro_provincia: t_registro_provincia1);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_provincia)
    else
        registro_provincia.codigo:=codigo_salida;
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_provincia: t_registro_provincia1;
    min: t_registro_provincia2;
    vector_provincias: t_vector_provincias;
    i: t_detalle;
    ok: boolean;
begin
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_provincia_detalle(vector_detalle[i],vector_provincias[i]);
        end;
    minimo(vector_detalle,vector_provincias,min);
    leer_provincia_maestro(archivo_maestro,registro_provincia);
    while (registro_provincia.codigo<>codigo_salida) do
        begin
            ok:=false;
            while (registro_provincia.codigo=min.codigo) do
                begin
                    ok:=true;
                    registro_provincia.kilos:=registro_provincia.kilos+min.kilos;
                    minimo(vector_detalle,vector_provincias,min);
                end;
            if (ok=true) then
                begin
                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                    write(archivo_maestro,registro_provincia);
                end;
            imprimir_texto(registro_provincia);
            leer_provincia_maestro(archivo_maestro,registro_provincia);
        end;
    close(archivo_maestro);
    for i:= 1 to detalles_total do
        close(vector_detalle[i]);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E8_provinciasMaestro');
    assign(archivo_carga_maestro,'E8_provinciasMaestro.txt');

```

```
cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
imprimir_archivo_maestro(archivo_maestro);
for i:= 1 to detalles_total do
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
    assign(vector_detalle[i],'E8_provinciasDetalle'+intToStr(i));
    assign(vector_carga_detalle[i],'E8_provinciasDetalle'+intToStr(i)+'.txt');
    cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
    imprimir_archivo_detalle(vector_detalle[i]);
end;
writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
//actualizar1_archivo_maestro(archivo_maestro,vector_detalle);
actualizar2_archivo_maestro(archivo_maestro,vector_detalle);
imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 9.

Se cuenta con un archivo que posee información de las ventas que realiza una empresa a los diferentes clientes. Se necesita obtener un reporte con las ventas organizadas por cliente. Para ello, se deberá informar por pantalla: los datos personales del cliente, el total mensual (mes por mes cuánto compró) y, finalmente, el monto total comprado en el año por el cliente. Además, al finalizar el reporte, se debe informar el monto total de ventas obtenido por la empresa.

El formato del archivo maestro está dado por: cliente (código cliente, nombre y apellido), año, mes, día y monto de la venta. El orden del archivo está dado por: código cliente, año y mes.

Nota: Tener en cuenta que puede haber meses en los que los clientes no realizaron compras. No es necesario informar tales meses en el reporte.

```
program TP2_E9;
{$codepage UTF8}
uses crt, sysutils;
const
  codigo_salida=999;
  dia_ini=1; dia_fin=31;
  mes_ini=1; mes_fin=12;
  anio_ini=2000; anio_fin=2025;
type
  t_string10=string[10];
  t_dia=dia_ini..dia_fin;
  t_mes=mes_ini..mes_fin;
  t_anio=anio_ini..anio_fin;
  t_registro_cliente=record
    codigo: int16;
    nombre: t_string10;
    apellido: t_string10;
  end;
  t_registro_fecha=record
    anio: t_anio;
    mes: t_mes;
    dia: t_dia;
  end;
  t_registro_venta=record
    cliente: t_registro_cliente;
    fecha: t_registro_fecha;
    monto: real;
  end;
  t_archivo_maestro=file of t_registro_venta;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_venta: t_registro_venta;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_venta do
      begin
        readln(archivo_carga_maestro,cliente.codigo,fecha.anio,fecha.mes,fecha.dia,monto,cliente
.nombre); cliente.nombre:=trim(cliente.nombre);
        readln(archivo_carga_maestro,cliente.apellido);
        write(archivo_maestro,registro_venta);
      end;
    end;
  end;
```

```

    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure imprimir_registro_cliente(registro_cliente: t_registro_cliente);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_cliente.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_cliente.nombre);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_cliente.apellido);
end;
procedure imprimir_registro_fecha(registro_fecha: t_registro_fecha);
begin
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_fecha.anio);
    textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.mes);
    textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.dia);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    imprimir_registro_cliente(registro_venta.cliente);
    imprimir_registro_fecha(registro_venta.fecha);
    textcolor(green); write('; Monto: $'); textcolor(yellow); writeln(registro_venta.monto:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_maestro);
end;
procedure leer_venta(var archivo_maestro: t_archivo_maestro; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_venta)
    else
        registro_venta.cliente.codigo:=codigo_salida;
end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_venta: t_registro_venta;
    monto_total, monto_anio, monto_mes: real;
    codigo, anio, mes: int16;
begin
    monto_total:=0;
    reset(archivo_maestro);
    leer_venta(archivo_maestro, registro_venta);
    while (registro_venta.cliente.codigo<>codigo_salida) do
        begin
            codigo:=registro_venta.cliente.codigo;
            textcolor(green); write('CLIENTE: '); imprimir_registro_cliente(registro_venta.cliente);
writeln(); writeln();
            while (registro_venta.cliente.codigo=codigo) do
                begin
                    anio:=registro_venta.fecha.anio;
                    monto_anio:=0;
                    textcolor(green); write('AÑO '); textcolor(yellow); writeln(anio);
                    while ((registro_venta.cliente.codigo=codigo) and (registro_venta.fecha.anio=anio)) do
                        begin
                            mes:=registro_venta.fecha.mes;
                            monto_mes:=0;

```

```
    while ((registro_venta.cliente.codigo=codigo) and (registro_venta.fecha.anio=anio) and
(registro_venta.fecha.mes=mes)) do
    begin
        monto_mes:=monto_mes+registro_venta.monto;
        leer_venta(archivo_maestro,registro_venta);
    end;
    textcolor(green); write('Monto total del mes '); textcolor(yellow); write(mes);
textcolor(green); write(': $'); textcolor(red); writeln(monto_mes:0:2);
    monto_anio:=monto_anio+monto_mes;
    end;
    textcolor(green); write('Monto total del año '); textcolor(yellow); write(anio);
textcolor(green); write(': $'); textcolor(red); writeln(monto_anio:0:2); writeln();
    monto_total:=monto_total+monto_anio;
    end;
end;
textcolor(green); write('Monto total de ventas obtenido por la empresa: $'); textcolor(red);
writeln(monto_total:0:2);
close(archivo_maestro);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E9_ventasMaestro');
assign(archivo_carga_maestro,'E9_ventasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    procesar_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 10.

Se necesita contabilizar los votos de las diferentes mesas electorales registradas por provincia y localidad. Para ello, se posee un archivo con la siguiente información: código de provincia, código de localidad, número de mesa y cantidad de votos en dicha mesa. Presentar en pantalla un listado como se muestra a continuación:

Código de Provincia

Código de Localidad

Total de Votos

.....

.....

.....

.....

Total de Votos Provincia: ____

Código de Provincia

Código de Localidad

Total de Votos

.....

.....

Total de Votos Provincia: ____

.....

Total General de Votos: ____

Nota: La información está ordenada por código de provincia y código de localidad.

```
program TP2_E10;
{$codepage UTF8}
uses crt;
const
    provincia_salida=999;
type
    t_registro_mesa=record
        provincia: int16;
        localidad: int16;
        numero: int16;
        votos: int16;
    end;
    t_archivo_maestro=file of t_registro_mesa;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_mesa: t_registro_mesa;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_mesa do
            begin
                readln(archivo_carga_maestro,provincia,localidad,numero,votos);
```

```

        write(archivo_maestro, registro_mesa);
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure imprimir_registro_mesa(registro_mesa: t_registro_mesa);
begin
    textcolor(green); write('Código de provincia: '); textcolor(yellow);
write(registro_mesa.provincia);
    textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_mesa.localidad);
    textcolor(green); write('; Número de mesa: '); textcolor(yellow);
write(registro_mesa.numero);
    textcolor(green); write('; Votos: '); textcolor(yellow); writeln(registro_mesa.votos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_mesa: t_registro_mesa;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_mesa);
            imprimir_registro_mesa(registro_mesa);
        end;
    close(archivo_maestro);
end;
procedure leer_mesa(var archivo_maestro: t_archivo_maestro; var registro_mesa:
t_registro_mesa);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_mesa)
    else
        registro_mesa.provincia:=provincia_salida;
end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_mesa: t_registro_mesa;
    provincia, localidad, votos_total, votos_provincia, votos_localidad: int16;
begin
    votos_total:=0;
    reset(archivo_maestro);
    leer_mesa(archivo_maestro, registro_mesa);
    while (registro_mesa.provincia<>provincia_salida) do
        begin
            provincia:=registro_mesa.provincia;
            votos_provincia:=0;
            textcolor(green); write('Código de Provincia: '); textcolor(yellow); writeln(provincia);
            textcolor(green); writeln('Código de Localidad          Total de Votos');
            while (registro_mesa.provincia=provincia) do
                begin
                    localidad:=registro_mesa.localidad;
                    votos_localidad:=0;
                    while ((registro_mesa.provincia=provincia) and (registro_mesa.localidad=localidad)) do
                        begin
                            votos_localidad:=votos_localidad+registro_mesa.votos;
                            leer_mesa(archivo_maestro, registro_mesa);
                        end;
                    textcolor(yellow); write(localidad); textcolor(green);
write('          '); textcolor(red); writeln(votos_localidad);
                    votos_provincia:=votos_provincia+votos_localidad;
                end;
                textcolor(green); write('Total de Votos Provincia: '); textcolor(red);
writeln(votos_provincia); writeln();
                votos_total:=votos_total+votos_provincia;
            end;
        end;
    end;
end;

```

```
    textcolor(green); write('Total General de Votos: '); textcolor(red); writeln(votos_total);
    close(archivo_maestro);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro, 'E10_mesasMaestro');
    assign(archivo_carga_maestro, 'E10_mesasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    procesar_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 11.

Se tiene información en un archivo de las horas extras realizadas por los empleados de una empresa en un mes. Para cada empleado, se tiene la siguiente información: departamento, división, número de empleado, categoría y cantidad de horas extras realizadas por el empleado. Se sabe que el archivo se encuentra ordenado por departamento, luego por división y, por último, por número de empleado. Presentar en pantalla un listado con el siguiente formato:

Departamento

División

Número de Empleado Total de Hs. Importe a cobrar

-----	-----	-----
-----	-----	-----

Total de horas división: ____

Monto total por división: ____

División

Total horas departamento: ____

Monto total departamento: ____

Para obtener el valor de la hora, se debe cargar un arreglo desde un archivo de texto al iniciar el programa con el valor de la hora extra para cada categoría. La categoría varía de 1 a 15. En el archivo de texto, debe haber una línea para cada categoría con el número de categoría y el valor de la hora, pero el arreglo debe ser de valores de horas, con la posición del valor coincidente con el número de categoría.

```
program TP2_E11;
{$codepage UTF8}
uses crt;
const
    departamento_salida=999;
    categoria_ini=1; categoria_fin=15;
type
    t_categoria=categoria_ini..categoria_fin;
    t_registro_empleado=record
        departamento: int16;
        division: int16;
        empleado: int16;
        categoria: t_categoria;
        horas: int16;
    end;
    t_vector_horas=array[t_categoria] of real;
    t_archivo_maestro=file of t_registro_empleado;
```

```
procedure cargar_vector_horas(var vector_horas: t_vector_horas; var archivo_carga_vector:
text);
var
    categoria: t_categoria;
    valor_hora: real;
begin
    reset(archivo_carga_vector);
    while (not eof(archivo_carga_vector)) do
        begin
            readln(archivo_carga_vector,categoria,valor_hora);
            vector_horas[categoria]:=valor_hora;
        end;
    close(archivo_carga_vector);
end;
procedure imprimir_vector_horas(vector_horas: t_vector_horas);
var
    i: t_categoria;
begin
    for i:= categoria_ini to categoria_fin do
        begin
            textcolor(green); write('El valor de la hora de la categoría ',i,' es $');
            textcolor(yellow); writeln(vector_horas[i]:0:2);
        end;
    end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_empleado: t_registro_empleado;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        begin
            with registro_empleado do
                begin
                    readln(archivo_carga_maestro,departamento,division,empleado,categoria,horas);
                    write(archivo_maestro,registro_empleado);
                end;
            end;
        close(archivo_maestro);
        close(archivo_carga_maestro);
    end;
procedure imprimir_registro_empleado(registro_empleado: t_registro_empleado);
begin
    textcolor(green); write('Departamento: '); textcolor(yellow);
write(registro_empleado.departamento);
    textcolor(green); write('; División: '); textcolor(yellow);
write(registro_empleado.division);
    textcolor(green); write('; Número de empleado: '); textcolor(yellow);
write(registro_empleado.empleado);
    textcolor(green); write('; Categoría: '); textcolor(yellow);
write(registro_empleado.categoria);
    textcolor(green); write('; Horas extras: '); textcolor(yellow);
writeln(registro_empleado.horas);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_empleado: t_registro_empleado;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_empleado);
            imprimir_registro_empleado(registro_empleado);
        end;
    close(archivo_maestro);
```

```

end;
procedure leer_empleado(var archivo_maestro: t_archivo_maestro; var registro_empleado:
t_registro_empleado);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_empleado)
    else
        registro_empleado.departamento:=departamento_salida;
    end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro; vector_horas:
t_vector_horas);
var
    registro_empleado: t_registro_empleado;
    categoria: t_categoria;
    departamento, division, empleado, horas_departamento, horas_division, horas_empleado: int16;
    monto_departamento, monto_division, monto_empleado: real;
begin
    reset(archivo_maestro);
    leer_empleado(archivo_maestro, registro_empleado);
    while (registro_empleado.departamento<>departamento_salida) do
        begin
            departamento:=registro_empleado.departamento;
            horas_departamento:=0; monto_departamento:=0;
            textcolor(green); write('Departamento: '); textcolor(yellow); writeln(departamento);
            while (registro_empleado.departamento=departamento) do
                begin
                    division:=registro_empleado.division;
                    horas_division:=0; monto_division:=0;
                    textcolor(green); write('División: '); textcolor(yellow); writeln(division);
                    textcolor(green); writeln('Número de Empleado      Total de Hs.      Importe a
cobrar');
                    while ((registro_empleado.departamento=departamento) and
(registro_empleado.division=division)) do
                        begin
                            empleado:=registro_empleado.empleado; categoria:=registro_empleado.categoria;
                            horas_empleado:=0; monto_empleado:=0;
                            while ((registro_empleado.departamento=departamento) and
(registro_empleado.division=division) and (registro_empleado.empleado=empleado)) do
                                begin
                                    horas_empleado:=horas_empleado+registro_empleado.horas;
                                    leer_empleado(archivo_maestro, registro_empleado);
                                end;
                                    monto_empleado:=horas_empleado*vector_horas[categoria];
                                    textcolor(yellow); write(empleado); textcolor(green);
write('
'); textcolor(red); write(horas_empleado); textcolor(green);
write('
    $'); textcolor(red); writeln(monto_empleado:0:2);
                                    horas_division:=horas_division+horas_empleado;
                                    monto_division:=monto_division+monto_empleado;
                                end;
                                    textcolor(green); write('Total horas división: '); textcolor(red);
writeln(horas_division);
                                    textcolor(green); write('Monto total división: $'); textcolor(red);
writeln(monto_division:0:2);
                                    horas_departamento:=horas_departamento+horas_division;
                                    monto_departamento:=monto_departamento+monto_division;
                                end;
                                    textcolor(green); write('Total horas departamento: '); textcolor(red);
writeln(horas_departamento);
                                    textcolor(green); write('Monto total departamento: $'); textcolor(red);
writeln(monto_departamento:0:2); writeln();
                                end;
                            end;
                        end;
                    end;
                end;
            end;
        end;
    var
        vector_horas: t_vector_horas;
        archivo_maestro: t_archivo_maestro;
        archivo_carga_vector, archivo_carga_maestro: text;

```

```
begin
  writeln(); textcolor(red); writeln('IMPRESIÓN VECTOR HORAS:'); writeln();
  assign(archivo_carga_vector, 'E11_horasVector.txt');
  cargar_vector_horas(vector_horas, archivo_carga_vector);
  imprimir_vector_horas(vector_horas);
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro, 'E11_empleadosMaestro');
  assign(archivo_carga_maestro, 'E11_empleadosMaestro.txt');
  cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
  imprimir_archivo_maestro(archivo_maestro);
  writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
  procesar_archivo_maestro(archivo_maestro, vector_horas);
end.
```

Ejercicio 12.

La empresa de software “X” posee un servidor web donde se encuentra alojado el sitio web de la organización. En dicho servidor, se almacenan, en un archivo, todos los accesos que se realizan al sitio. La información que se almacena en el archivo es la siguiente: año, mes, día, idUsuario y tiempo de acceso al sitio de la organización. El archivo se encuentra ordenado por los siguientes criterios: año, mes, día e idUsuario.

Se debe realizar un procedimiento que genere un informe en pantalla. Para ello, se indicará el año calendario sobre el cual debe realizar el informe. El mismo debe respetar el formato mostrado a continuación:

```
Año : ---
  Mes:-- 1
    día:-- 1
      idUsuario 1  Tiempo Total de acceso en el día 1 mes 1
      -----
      idUsuario N  Tiempo total de acceso en el día 1 mes 1
      Tiempo total acceso día 1 mes 1
    -----
  día N
    idUsuario 1  Tiempo Total de acceso en el día N mes 1
    -----
    idUsuario N  Tiempo total de acceso en el día N mes 1
    Tiempo total acceso día N mes 1
  Total tiempo de acceso mes 1
  -----
Mes 12
  día 1
    idUsuario 1  Tiempo Total de acceso en el día 1 mes 12
    -----
    idUsuario N  Tiempo total de acceso en el día 1 mes 12
    Tiempo total acceso día 1 mes 12
  -----
  día N
    idUsuario 1  Tiempo Total de acceso en el día N mes 12
    -----
    idUsuario N  Tiempo total de acceso en el día N mes 12
    Tiempo total acceso día N mes 12
  Total tiempo de acceso mes 12
Total tiempo de acceso año
```

Se deberá tener en cuenta las siguientes aclaraciones:

- El año sobre el cual realizará el informe de accesos debe leerse desde el teclado.

- El año puede no existir en el archivo, en tal caso debe informarse en pantalla “Año no encontrado”.
- Se debe definir las estructuras de datos necesarias.
- El recorrido del archivo debe realizarse una única vez, procesando sólo la información necesaria.

```

program TP2_E12;
{$codepage UTF8}
uses crt;
const
  anio_salida=999;
  dia_ini=1; dia_fin=31;
  mes_ini=1; mes_fin=12;
  anio_ini=2020; anio_fin=2025;
type
  t_dia=dia_ini..dia_fin;
  t_mes=mes_ini..mes_fin;
  t_anio=anio_ini..anio_fin;
  t_registro_fecha=record
    anio: int16;
    mes: t_mes;
    dia: t_dia;
  end;
  t_registro_usuario=record
    fecha: t_registro_fecha;
    usuario: int16;
    tiempo: real;
  end;
  t_archivo_maestro=file of t_registro_usuario;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_usuario: t_registro_usuario;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_usuario do
      begin
        readln(archivo_carga_maestro, fecha.anio, fecha.mes, fecha.dia, usuario, tiempo);
        write(archivo_maestro, registro_usuario);
      end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
  end;
procedure imprimir_registro_fecha(registro_fecha: t_registro_fecha);
begin
  textcolor(green); write('Fecha: '); textcolor(yellow); write(registro_fecha.anio);
  textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.mes);
  textcolor(green); write('/'); textcolor(yellow); write(registro_fecha.dia);
end;
procedure imprimir_registro_usuario(registro_usuario: t_registro_usuario);
begin
  imprimir_registro_fecha(registro_usuario.fecha);
  textcolor(green); write('; ID usuario: '); textcolor(yellow);
write(registro_usuario.usuario);
  textcolor(green); write('; Tiempo de acceso: '); textcolor(yellow);
writeln(registro_usuario.tiempo:0:2);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_usuario: t_registro_usuario;
begin

```

```

reset(archivo_maestro);
while (not eof(archivo_maestro)) do
begin
    read(archivo_maestro,registro_usuario);
    imprimir_registro_usuario(registro_usuario);
end;
close(archivo_maestro);
end;
procedure leer_usuario(var archivo_maestro: t_archivo_maestro; var registro_usuario:
t_registro_usuario);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_usuario)
    else
        registro_usuario.fecha.anio:=anio_salida;
    end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro; anio: t_anio);
var
    registro_usuario: t_registro_usuario;
    mes, dia, usuario: int16;
    tiempo_anio, tiempo_mes, tiempo_dia, tiempo_usuario: real;
begin
    reset(archivo_maestro);
    leer_usuario(archivo_maestro,registro_usuario);
    while ((registro_usuario.fecha.anio<>anio) and (registro_usuario.fecha.anio<>anio_salida))
do
    leer_usuario(archivo_maestro,registro_usuario);
    if (registro_usuario.fecha.anio<>anio_salida) then
    begin
        tiempo_anio:=0;
        textcolor(green); write('Año: '); textcolor(yellow); writeln(anio); writeln();
        while (registro_usuario.fecha.anio=anio) do
        begin
            mes:=registro_usuario.fecha.mes;
            tiempo_mes:=0;
            textcolor(green); write(' Mes: '); textcolor(yellow); writeln(mes);
            while ((registro_usuario.fecha.anio=anio) and (registro_usuario.fecha.mes=mes)) do
            begin
                dia:=registro_usuario.fecha.dia;
                tiempo_dia:=0;
                textcolor(green); write(' Día: '); textcolor(yellow); writeln(dia);
                textcolor(green); write(' idUsuario Tiempo Total de acceso en el día ');
                textcolor(yellow); write(dia); textcolor(green); write(' mes '); textcolor(yellow);
writeln(mes);
                while ((registro_usuario.fecha.anio=anio) and (registro_usuario.fecha.mes=mes) and
(registro_usuario.fecha.dia=dia)) do
                begin
                    usuario:=registro_usuario.usuario;
                    tiempo_usuario:=0;
                    while ((registro_usuario.fecha.anio=anio) and (registro_usuario.fecha.mes=mes) and
(registro_usuario.fecha.dia=dia) and (registro_usuario.usuario=usuario)) do
                    begin
                        tiempo_usuario:=tiempo_usuario+registro_usuario.tiempo;
                        leer_usuario(archivo_maestro,registro_usuario);
                    end;
                    textcolor(green); write(' '); textcolor(yellow); write(usuario);
                    textcolor(green); write(' '); textcolor(red); writeln(tiempo_usuario:0:2);
                    tiempo_dia:=tiempo_dia+tiempo_usuario;
                end;
                textcolor(green); write(' Tiempo total acceso día '); textcolor(yellow);
write(dia); textcolor(green); write(' mes '); textcolor(yellow); write(mes); textcolor(green);
write(': '); textcolor(red); writeln(tiempo_dia:0:2);
                tiempo_mes:=tiempo_mes+tiempo_dia;
            end;
            textcolor(green); write(' Tiempo total acceso mes '); textcolor(yellow); write(mes);
                    textcolor(green); write(': '); textcolor(red); writeln(tiempo_mes:0:2); writeln();
        end;
    end;
end;

```

```
        tiempo_anio:=tiempo_anio+tiempo_mes;
    end;
    textcolor(green); write('Tiempo total acceso año '); textcolor(yellow); write(anio);
textcolor(green); write(': '); textcolor(red); writeln(tiempo_anio:0:2);
end
else
begin
    textcolor(green); write('Año '); textcolor(yellow); write(anio); textcolor(green);
writeln(' no encontrado');
end;
close(archivo_maestro);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    anio: t_anio;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E12_usuariosMaestro');
assign(archivo_carga_maestro,'E12_usuariosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    anio:=anio_ini+random(anio_fin-anio_ini+1);
    procesar_archivo_maestro(archivo_maestro,anio);
end.
```

Ejercicio 13.

Suponer que se es administrador de un servidor de correo electrónico. En los logs del mismo (información guardada acerca de los movimientos que ocurren en el server) que se encuentran en la ruta `/var/log/logmail.dat`, se guarda la siguiente información: `nro_usuario`, `nombreUsuario`, `nombre`, `apellido`, `cantidadMailEnviados`. Diariamente, el servidor de correo genera un archivo con la siguiente información: `nro_usuario`, `cuentaDestino`, `cuerpoMensaje`. Este archivo representa todos los correos enviados por los usuarios en un día determinado. Ambos archivos están ordenados por `nro_usuario` y se sabe que un usuario puede enviar cero, uno o más mails por día.

(a) Realizar el procedimiento necesario para actualizar la información del log en un día particular. Definir las estructuras de datos que utilice el procedimiento.

(b) Generar un archivo de texto que contenga el siguiente informe dado un archivo detalle de un día determinado:

`nro_usuarioX.....cantidadMensajesEnviados`

.....

`nro_usuarioX+n.....cantidadMensajesEnviados`

Nota: Tener en cuenta que, en el listado, deberán aparecer todos los usuarios que existen en el sistema. Considerar la implementación de esta opción de las siguientes maneras:

- Como un procedimiento separado del inciso (a).
- En el mismo procedimiento de actualización del inciso (a). ¿Qué cambios se requieren en el procedimiento del inciso (a) para realizar el informe en el mismo recorrido?

```
program TP2_E13;
{$codepage UTF8}
uses crt, sysutils;
const
    usuario_salida=999;
type
    t_string20=string[20];
    t_registro_usuario1=record
        usuario: int16;
        nombre_usuario: t_string20;
        nombre: t_string20;
        apellido: t_string20;
        mails: int16;
    end;
    t_registro_usuario2=record
        usuario: int16;
        destino: t_string20;
        mensaje: t_string20;
    end;
    t_archivo_maestro=file of t_registro_usuario1;
    t_archivo_detalle=file of t_registro_usuario2;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
```

```

registro_usuario: t_registro_usuario1;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_usuario do
      begin
        readln(archivo_carga_maestro, usuario, mails, nombre_usuario);
nombre_usuario:=trim(nombre_usuario);
        readln(archivo_carga_maestro, nombre);
        readln(archivo_carga_maestro, apellido);
        write(archivo_maestro, registro_usuario);
      end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
  end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
  registro_usuario: t_registro_usuario2;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_usuario do
      begin
        readln(archivo_carga_detalle, usuario, destino); destino:=trim(destino);
        readln(archivo_carga_detalle, mensaje);
        write(archivo_detalle, registro_usuario);
      end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
  end;
procedure imprimir_registro_usuario1(registro_usuario: t_registro_usuario1);
begin
  textcolor(green); write('Número de usuario: '); textcolor(yellow);
write(registro_usuario.usuario);
  textcolor(green); write('; Nombre de usuario: '); textcolor(yellow);
write(registro_usuario.nombre_usuario);
  textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_usuario.nombre);
  textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_usuario.apellido);
  textcolor(green); write('; Mails enviados: '); textcolor(yellow);
writeln(registro_usuario.mails);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
  registro_usuario: t_registro_usuario1;
begin
  reset(archivo_maestro);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro, registro_usuario);
      imprimir_registro_usuario1(registro_usuario);
    end;
  close(archivo_maestro);
end;
procedure imprimir_registro_usuario2(registro_usuario: t_registro_usuario2);
begin
  textcolor(green); write('Número de usuario: '); textcolor(yellow);
write(registro_usuario.usuario);
  textcolor(green); write('; Cuenta destino: '); textcolor(yellow);
write(registro_usuario.destino);
  textcolor(green); write('; Cuerpo del mensaje: '); textcolor(yellow);
writeln(registro_usuario.mensaje);
end;

```

```

procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_usuario: t_registro_usuario2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle, registro_usuario);
            imprimir_registro_usuario2(registro_usuario);
        end;
    close(archivo_detalle);
end;

procedure leer_usuario(var archivo_detalle: t_archivo_detalle; var registro_usuario:
t_registro_usuario2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_usuario)
    else
        registro_usuario.usuario:=usuario_salida;
    end;

procedure imprimir_texto(registro_maestro: t_registro_usuario1);
begin
    textcolor(green); write('Número de Usuario: '); textcolor(yellow);
write(registro_maestro.usuario); textcolor(green); write(' ..... Cantidad de Mensajes
Enviados: '); textcolor(red); writeln(registro_maestro.mails);
end;

procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
    registro_usuario: t_registro_usuario1;
    archivo_txt: text;
begin
    reset(archivo_maestro);
    assign(archivo_txt, 'E13_usuarios.txt'); rewrite(archivo_txt);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_usuario);
            writeln(archivo_txt, registro_usuario.usuario, ' ', registro_usuario.mails);
        end;
    close(archivo_txt);
end;

procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_detalle: t_archivo_detalle);
var
    registro_maestro: t_registro_usuario1;
    registro_detalle: t_registro_usuario2;
begin
    reset(archivo_maestro);
    reset(archivo_detalle);
    leer_usuario(archivo_detalle, registro_detalle);
    while (registro_detalle.usuario<>usuario_salida) do
        begin
            read(archivo_maestro, registro_maestro);
            while (registro_maestro.usuario<>registro_detalle.usuario) do
                begin
                    imprimir_texto(registro_maestro);
                    read(archivo_maestro, registro_maestro);
                end;
            while (registro_maestro.usuario=registro_detalle.usuario) do
                begin
                    registro_maestro.mails:=registro_maestro.mails+1;
                    leer_usuario(archivo_detalle, registro_detalle);
                end;
            seek(archivo_maestro, filepos(archivo_maestro)-1);
            write(archivo_maestro, registro_maestro);
            imprimir_texto(registro_maestro);
        end;
    end;
end;

```

```
while (not eof(archivo_maestro)) do
begin
    read(archivo_maestro,registro_maestro);
    imprimir_texto(registro_maestro);
end;
exportar_archivo_txt(archivo_maestro);
close(archivo_maestro);
close(archivo_detalle);
writeln();
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_detalle: t_archivo_detalle;
    archivo_carga_maestro, archivo_carga_detalle: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E13_usuariosMaestro');
    assign(archivo_carga_maestro,'E13_usuariosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE:'); writeln();
    assign(archivo_detalle,'E13_usuariosDetalle');
    assign(archivo_carga_detalle,'E13_usuariosDetalle.txt');
    cargar_archivo_detalle(archivo_detalle,archivo_carga_detalle);
    imprimir_archivo_detalle(archivo_detalle);
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,archivo_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 14.

Una compañía aérea dispone de un archivo maestro donde guarda información sobre sus próximos vuelos. En dicho archivo, se tiene almacenado el destino, fecha, hora de salida y la cantidad de asientos disponibles. La empresa recibe todos los días dos archivos detalles para actualizar el archivo maestro. En dichos archivos, se tiene destino, fecha, hora de salida y cantidad de asientos comprados. Se sabe que los archivos están ordenados por destino más fecha y hora de salida, y que, en los detalles, pueden venir 0, 1 o más registros por cada uno del maestro. Se pide realizar los módulos necesarios para:

- (a) Actualizar el archivo maestro sabiendo que no se registró ninguna venta de pasaje sin asiento disponible.
- (b) Generar una lista con aquellos vuelos (destino, fecha y hora de salida) que tengan menos de una cantidad específica de asientos disponibles. La misma debe ser ingresada por teclado.

Nota: El archivo maestro y los archivos detalles sólo pueden recorrerse una vez.

```
program TP2_E14;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=2;
    destino_salida='ZZZ';
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_vuelo=record
        destino: t_string20;
        fecha: t_string20;
        hora: t_string20;
        asientos_disponibles: int16;
    end;
    t_registro_venta=record
        destino: t_string20;
        fecha: t_string20;
        hora: t_string20;
        asientos_vendidos: int16;
    end;
    t_archivo_maestro=file of t_registro_vuelo;
    t_archivo_detalle=file of t_registro_venta;
    t_vector_ventas=array[t_detalle] of t_registro_venta;
    t_vector_detalle=array[t_detalle] of t_archivo_detalle;
    t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_vuelo: t_registro_vuelo;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_vuelo do
            begin
                readln(archivo_carga_maestro,asientos_disponibles,destino); destino:=trim(destino);
                readln(archivo_carga_maestro,fecha);
```

```

        readln(archivo_carga_maestro,hora);
        write(archivo_maestro,registro_vuelo);
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,asientos_vendidos,destino); destino:=trim(destino);
                readln(archivo_carga_detalle,fecha);
                readln(archivo_carga_detalle,hora);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_vuelo(registro_vuelo: t_registro_vuelo);
begin
    textcolor(green); write('Destino: '); textcolor(yellow); write(registro_vuelo.destino);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_vuelo.fecha);
    textcolor(green); write('; Hora: '); textcolor(yellow); write(registro_vuelo.hora);
    textcolor(green); write('; Asientos disponibles: '); textcolor(yellow);
writeln(registro_vuelo.asientos_disponibles);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_vuelo: t_registro_vuelo;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_vuelo);
            imprimir_registro_vuelo(registro_vuelo);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Destino: '); textcolor(yellow); write(registro_venta.destino);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_venta.fecha);
    textcolor(green); write('; Hora: '); textcolor(yellow); write(registro_venta.hora);
    textcolor(green); write('; Asientos vendidos: '); textcolor(yellow);
writeln(registro_venta.asientos_vendidos);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);

```

```

begin
  if (not eof(archivo_detalle)) then
    read(archivo_detalle, registro_venta)
  else
    registro_venta.destino:=destino_salida;
end;
procedure minimo(var vector_detalles: t_vector_detalles; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);
var
  i, pos: t_detalle;
begin
  min.destino:=destino_salida;
  for i:= 1 to detalles_total do
    if ((vector_ventas[i].destino<min.destino) or ((vector_ventas[i].destino=min.destino) and
(vector_ventas[i].fecha<min.fecha)) or ((vector_ventas[i].destino=min.destino) and
(vector_ventas[i].fecha=min.fecha) and (vector_ventas[i].hora<min.hora))) then
      begin
        min:=vector_ventas[i];
        pos:=i;
      end;
    if (min.destino<destino_salida) then
      leer_venta(vector_detalles[pos], vector_ventas[pos]);
end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalles: t_vector_detalles; asientos: int16);
var
  registro_vuelo: t_registro_vuelo;
  min: t_registro_venta;
  vector_ventas: t_vector_ventas;
  i: t_detalle;
begin
  reset(archivo_maestro);
  for i:= 1 to detalles_total do
    begin
      reset(vector_detalles[i]);
      leer_venta(vector_detalles[i], vector_ventas[i]);
    end;
    minimo(vector_detalles, vector_ventas, min);
    while (min.destino<>destino_salida) do
      begin
        read(archivo_maestro, registro_vuelo);
        while (registro_vuelo.destino<>min.destino) do
          read(archivo_maestro, registro_vuelo);
        while (registro_vuelo.destino=min.destino) do
          begin
            while (registro_vuelo.fecha<>min.fecha) do
              read(archivo_maestro, registro_vuelo);
            while ((registro_vuelo.destino=min.destino) and (registro_vuelo.fecha=min.fecha)) do
              begin
                while (registro_vuelo.hora<>min.hora) do
                  read(archivo_maestro, registro_vuelo);
                while ((registro_vuelo.destino=min.destino) and (registro_vuelo.fecha=min.fecha) and
(registro_vuelo.hora=min.hora)) do
                  begin
                    registro_vuelo.asientos_disponibles:=registro_vuelo.asientos_disponibles-
min.asientos_vendidos;
                    minimo(vector_detalles, vector_ventas, min);
                  end;
                  seek(archivo_maestro, filepos(archivo_maestro)-1);
                  write(archivo_maestro, registro_vuelo);
                  if (registro_vuelo.asientos_disponibles<asientos) then
                    begin
                      textcolor(green); write('El vuelo con destino a '); textcolor(yellow);
write(registro_vuelo.destino); textcolor(green); write(' del '); textcolor(yellow);
write(registro_vuelo.fecha); textcolor(green); write(' a las '); textcolor(yellow);

```

```

write(registro_vuelo.hora); textcolor(green); write(' tiene menos de '); textcolor(yellow);
write(asientos); textcolor(green); writeln(' asientos disponibles');
    end;
    end;
    end;
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
    asientos: int16;
begin
    randomize;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E14_vuelosMaestro');
    assign(archivo_carga_maestro,'E14_vuelosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i],'E14_ventasDetalle'+inttoStr(i));
            assign(vector_carga_detalle[i],'E14_ventasDetalle'+inttoStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    asientos:=10+random(291);
    actualizar_archivo_maestro(archivo_maestro,vector_detalle,asientos);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 15.

Se desea modelar la información de una ONG dedicada a la asistencia de personas con carencias habitacionales. La ONG cuenta con un archivo maestro conteniendo información como se indica a continuación: código provincia, nombre provincia, código de localidad, nombre de localidad, #viviendas sin luz, #viviendas sin gas, #viviendas de chapa, #viviendas sin agua, #viviendas sin sanitarios.

Mensualmente, reciben detalles de las diferentes provincias indicando avances en las obras de ayuda en la edificación y equipamientos de viviendas en cada provincia. La información de los detalles es la siguiente: código provincia, código localidad, #viviendas con luz, #viviendas construidas, #viviendas con agua, #viviendas con gas, #entrega sanitarios.

Se debe realizar el procedimiento que permita actualizar el maestro con los detalles recibidos, se reciben 10 detalles. Todos los archivos están ordenados por código de provincia y código de localidad.

Para la actualización del archivo maestro, se debe proceder de la siguiente manera:

- *Al valor de viviendas sin luz se le resta el valor recibido en el detalle.*
- *Ídem para viviendas sin agua, sin gas y sin sanitarios.*
- *A las viviendas de chapa se le resta el valor recibido de viviendas construidas.*

La misma combinación de provincia y localidad aparecen, a lo sumo, una única vez.

Realizar las declaraciones necesarias, el programa principal y los procedimientos que se requiera para la actualización solicitada e informar cantidad de localidades sin viviendas de chapa (las localidades pueden o no haber sido actualizadas).

```
program TP2_E15;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=10;
  codigo_salida=999;
  viviendas_de_chapa_corte=0;
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_provincia1=record
    codigo: int16;
    nombre: t_string20;
    codigo_localidad: int16;
    nombre_localidad: t_string20;
    viviendas_sin_luz: int16;
    viviendas_sin_gas: int16;
    viviendas_de_chapa: int16;
    viviendas_sin_agua: int16;
    viviendas_sin_sanitarios: int16;
  end;
  t_registro_provincia2=record
    codigo: int16;
    codigo_localidad: int16;
    viviendas_con_luz: int16;
```

```

viviendas_construidas: int16;
viviendas_con_agua: int16;
viviendas_con_gas: int16;
entrega_sanitarios: int16;
end;
t_archivo_maestro=file of t_registro_provincia1;
t_archivo_detalle=file of t_registro_provincia2;
t_vector_provincias=array[t_detalle] of t_registro_provincia2;
t_vector_detalle=array[t_detalle] of t_archivo_detalle;
t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_provincia: t_registro_provincia1;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_provincia do
            begin
                readln(archivo_carga_maestro,codigo,codigo_localidad,viviendas_sin_luz,viviendas_sin_gas
,viviendas_de_chapa,viviendas_sin_agua,viviendas_sin_sanitarios,nombre); nombre:=trim(nombre);
                readln(archivo_carga_maestro,nombre_localidad);
nombre_localidad:=trim(nombre_localidad);
                write(archivo_maestro,registro_provincia);
                end;
            close(archivo_maestro);
            close(archivo_carga_maestro);
        end;
    procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
    var
        registro_provincia: t_registro_provincia2;
    begin
        rewrite(archivo_detalle);
        reset(archivo_carga_detalle);
        while (not eof(archivo_carga_detalle)) do
            with registro_provincia do
                begin
                    readln(archivo_carga_detalle,codigo,codigo_localidad,viviendas_con_luz,viviendas_constru
idas,viviendas_con_agua,viviendas_con_gas,entrega_sanitarios);
                    write(archivo_detalle,registro_provincia);
                    end;
                close(archivo_detalle);
                close(archivo_carga_detalle);
            end;
        procedure imprimir_registro_provincial(registro_provincia: t_registro_provincia1);
        begin
            textcolor(green); write('Código de provincia: '); textcolor(yellow);
write(registro_provincia.codigo);
            textcolor(green); write('; Nombre de provincia: '); textcolor(yellow);
write(registro_provincia.nombre);
            textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_provincia.codigo_localidad);
            textcolor(green); write('; Nombre de localidad: '); textcolor(yellow);
write(registro_provincia.nombre_localidad);
            textcolor(green); write('; Viviendas sin luz: '); textcolor(yellow);
write(registro_provincia.viviendas_sin_luz);
            textcolor(green); write('; Viviendas sin gas: '); textcolor(yellow);
write(registro_provincia.viviendas_sin_gas);
            textcolor(green); write('; Viviendas de chapa: '); textcolor(yellow);
write(registro_provincia.viviendas_de_chapa);
            textcolor(green); write('; Viviendas sin agua: '); textcolor(yellow);
write(registro_provincia.viviendas_sin_agua);
            textcolor(green); write('; Viviendas sin sanitarios: '); textcolor(yellow);
writeln(registro_provincia.viviendas_sin_sanitarios);

```

```

end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_provincia: t_registro_provincia1;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_provincia);
            imprimir_registro_provincia1(registro_provincia);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_provincia2(registro_provincia: t_registro_provincia2);
begin
    textcolor(green); write('Código de provincia: '); textcolor(yellow);
write(registro_provincia.codigo);
    textcolor(green); write('; Código de localidad: '); textcolor(yellow);
write(registro_provincia.codigo_localidad);
    textcolor(green); write('; Viviendas con luz: '); textcolor(yellow);
write(registro_provincia.viviendas_con_luz);
    textcolor(green); write('; Viviendas construidas: '); textcolor(yellow);
write(registro_provincia.viviendas_construidas);
    textcolor(green); write('; Viviendas con agua: '); textcolor(yellow);
write(registro_provincia.viviendas_con_agua);
    textcolor(green); write('; Viviendas con gas: '); textcolor(yellow);
write(registro_provincia.viviendas_con_gas);
    textcolor(green); write('; Entrega sanitarios: '); textcolor(yellow);
writeln(registro_provincia.entrega_sanitarios);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_provincia: t_registro_provincia2;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle, registro_provincia);
            imprimir_registro_provincia2(registro_provincia);
        end;
    close(archivo_detalle);
end;
procedure leer_provincia_detalle(var archivo_detalle: t_archivo_detalle; var
registro_provincia: t_registro_provincia2);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_provincia)
    else
        registro_provincia.codigo:=codigo_salida;
end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_provincias:
t_vector_provincias; var min: t_registro_provincia2);
var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida;
    for i:= 1 to detalles_total do
        if ((vector_provincias[i].codigo<min.codigo) or ((vector_provincias[i].codigo=min.codigo)
and (vector_provincias[i].codigo_localidad<min.codigo_localidad))) then
            begin
                min:=vector_provincias[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida) then
            leer_provincia_detalle(vector_detalle[pos], vector_provincias[pos]);
        end;
    end;
end;

```

```

procedure actualizar_localidades_corte(viviendas_de_chapa_localidad: int16; var
localidades_corte: int16);
begin
    if (viviendas_de_chapa_localidad=viviendas_de_chapa_corte) then
        localidades_corte:=localidades_corte+1;
    end;
procedure actualizar1_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_provincia: t_registro_provincia1;
    min: t_registro_provincia2;
    vector_provincias: t_vector_provincias;
    i: t_detalle;
    localidades_corte: int16;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_provincia_detalle(vector_detalle[i],vector_provincias[i]);
        end;
        minimo(vector_detalle,vector_provincias,min);
        while (min.codigo<>codigo_salida) do
            begin
                read(archivo_maestro,registro_provincia);
                while (registro_provincia.codigo<>min.codigo) do
                    begin
                        actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
                        read(archivo_maestro,registro_provincia);
                    end;
                    ok:=true;
                    while (registro_provincia.codigo=min.codigo) do
                        begin
                            while (registro_provincia.codigo_localidad<>min.codigo_localidad) do
                                begin
                                    if (ok=true) then
                                        actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte
);
                                    read(archivo_maestro,registro_provincia);
                                    ok:=true;
                                end;
                                while ((registro_provincia.codigo=min.codigo) and
(registro_provincia.codigo_localidad=min.codigo_localidad)) do
                                    begin
                                        registro_provincia.viviendas_sin_luz:=registro_provincia.viviendas_sin_luz-
min.viviendas_con_luz;
                                        registro_provincia.viviendas_sin_agua:=registro_provincia.viviendas_sin_agua-
min.viviendas_con_agua;
                                        registro_provincia.viviendas_sin_gas:=registro_provincia.viviendas_sin_gas-
min.viviendas_con_gas;
                                        registro_provincia.viviendas_sin_sanitarios:=registro_provincia.viviendas_sin_sanitari
os-min.entrega_sanitarios;
                                        registro_provincia.viviendas_de_chapa:=registro_provincia.viviendas_de_chapa-
min.viviendas_construidas;
                                        minimo(vector_detalle,vector_provincias,min);
                                    end;
                                    seek(archivo_maestro,filepos(archivo_maestro)-1);
                                    write(archivo_maestro,registro_provincia);
                                    actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
                                    ok:=false;
                                end;
                            end;
                        while (not eof(archivo_maestro)) do
                            begin

```

```

    read(archivo_maestro,registro_provincia);
    actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con '); textcolor(yellow);
write(viviendas_de_chapa_corte); textcolor(green); write(' viviendas de chapa es ');
textcolor(red); writeln(localidades_corte);
    writeln();
end;
procedure leer_provincia_maestro(var archivo_maestro: t_archivo_maestro; var
registro_provincia: t_registro_provincial);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro,registro_provincia)
    else
        registro_provincia.codigo:=codigo_salida;
end;
procedure actualizar2_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_provincia: t_registro_provincial;
    min: t_registro_provincia2;
    vector_provincias: t_vector_provincias;
    i: t_detalle;
    localidades_corte: int16;
    ok: boolean;
begin
    localidades_corte:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_provincia_detalle(vector_detalle[i],vector_provincias[i]);
        end;
        minimo(vector_detalle,vector_provincias,min);
        leer_provincia_maestro(archivo_maestro,registro_provincia);
        while (registro_provincia.codigo<>codigo_salida) do
            begin
                ok:=false;
                while ((registro_provincia.codigo=min.codigo) and
(registro_provincia.codigo_localidad=min.codigo_localidad)) do
                    begin
                        ok:=true;
                        registro_provincia.viviendas_sin_luz:=registro_provincia.viviendas_sin_luz-
min.viviendas_con_luz;
                        registro_provincia.viviendas_sin_agua:=registro_provincia.viviendas_sin_agua-
min.viviendas_con_agua;
                        registro_provincia.viviendas_sin_gas:=registro_provincia.viviendas_sin_gas-
min.viviendas_con_gas;
                        registro_provincia.viviendas_sin_sanitarios:=registro_provincia.viviendas_sin_sanitarios-
min.entrega_sanitarios;
                        registro_provincia.viviendas_de_chapa:=registro_provincia.viviendas_de_chapa-
min.viviendas_construidas;
                        minimo(vector_detalle,vector_provincias,min);
                    end;
                    if (ok=true) then
                        begin
                            seek(archivo_maestro,filepos(archivo_maestro)-1);
                            write(archivo_maestro,registro_provincia);
                        end;
                        actualizar_localidades_corte(registro_provincia.viviendas_de_chapa,localidades_corte);
                        leer_provincia_maestro(archivo_maestro,registro_provincia);
                    end;
                end;
            end;
        close(archivo_maestro);

```

```
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('La cantidad de localidades con '); textcolor(yellow);
write(viviendas_de_chapa_corte); textcolor(green); write(' viviendas de chapa es ');
textcolor(red); writeln(localidades_corte);
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E15_provinciasMaestro');
assign(archivo_carga_maestro,'E15_provinciasMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,':'); writeln();
            assign(vector_detalle[i],'E15_provinciasDetalle'+intToStr(i));
assign(vector_carga_detalle[i],'E15_provinciasDetalle'+intToStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    //actualizar1_archivo_maestro(archivo_maestro,vector_detalle);
    actualizar2_archivo_maestro(archivo_maestro,vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 16.

La editorial X, autora de diversos semanarios, posee un archivo maestro con la información correspondiente a las diferentes emisiones de los mismos. De cada emisión, se registra: fecha, código de semanario, nombre del semanario, descripción, precio, total de ejemplares y total de ejemplares vendidos.

Mensualmente, se reciben 100 archivos detalles con las ventas de los semanarios en todo el país. La información que poseen los detalles es la siguiente: fecha, código de semanario y cantidad de ejemplares vendidos. Realizar las declaraciones necesarias, la llamada al procedimiento y el procedimiento que recibe el archivo maestro y los 100 detalles y realizar la actualización del archivo maestro en función de las ventas registradas. Además, se deberá informar fecha y semanario que tuvo más ventas y la misma información del semanario con menos ventas.

Nota: Todos los archivos están ordenados por fecha y código de semanario. No se realizan ventas de semanarios si no hay ejemplares para hacerlo.

```
program TP2_E16;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_total=3; // detalles_total=100;
  fecha_salida='ZZZ';
type
  t_detalle=1..detalles_total;
  t_string20=string[20];
  t_registro_semanario=record
    fecha: t_string20;
    codigo: int16;
    nombre: t_string20;
    descripcion: t_string20;
    precio: real;
    total_ejemplares: int16;
    ejemplares_vendidos: int16;
  end;
  t_registro_venta=record
    fecha: t_string20;
    codigo: int16;
    ejemplares_vendidos: int16;
  end;
  t_archivo_maestro=file of t_registro_semanario;
  t_archivo_detalle=file of t_registro_venta;
  t_vector_ventas=array[t_detalle] of t_registro_venta;
  t_vector_detalle=array[t_detalle] of t_archivo_detalle;
  t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
  registro_semanario: t_registro_semanario;
begin
  rewrite(archivo_maestro);
  reset(archivo_carga_maestro);
  while (not eof(archivo_carga_maestro)) do
    with registro_semanario do
      begin
        readln(archivo_carga_maestro,codigo,precio,total_ejemplares,ejemplares_vendidos,fecha);
        fecha:=trim(fecha);
        readln(archivo_carga_maestro,nombre); nombre:=trim(nombre);
```

```

        readln(archivo_carga_maestro,descripcion); descripcion:=trim(descripcion);
        write(archivo_maestro,registro_semanario);
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,ejemplares_vendidos,fecha); fecha:=trim(fecha);
                write(archivo_detalle,registro_venta);
            end;
        end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_semanario(registro_semanario: t_registro_semanario);
begin
    textcolor(green); write('Fecha: '); textcolor(yellow); write(registro_semanario.fecha);
    textcolor(green); write('; Código: '); textcolor(yellow); write(registro_semanario.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_semanario.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
    write(registro_semanario.descripcion);
    textcolor(green); write('; Precio: $'); textcolor(yellow);
    write(registro_semanario.precio:0:2);
    textcolor(green); write('; Total de ejemplares: '); textcolor(yellow);
    write(registro_semanario.total_ejemplares);
    textcolor(green); write('; Ejemplares vendidos: '); textcolor(yellow);
    writeln(registro_semanario.ejemplares_vendidos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_semanario: t_registro_semanario;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_semanario);
            imprimir_registro_semanario(registro_semanario);
        end;
    close(archivo_maestro);
end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Fecha: '); textcolor(yellow); write(registro_venta.fecha);
    textcolor(green); write('; Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Ejemplares vendidos: '); textcolor(yellow);
    writeln(registro_venta.ejemplares_vendidos);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
    close(archivo_detalle);
end;

```

```

end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_venta)
    else
        registro_venta.fecha:=fecha_salida;
    end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);
var
    i, pos: t_detalle;
begin
    min.fecha:=fecha_salida;
    for i:= 1 to detalles_total do
        if ((vector_ventas[i].fecha<min.fecha) or ((vector_ventas[i].fecha=min.fecha) and
(vector_ventas[i].codigo<min.codigo))) then
            begin
                min:=vector_ventas[i];
                pos:=i;
            end;
        if (min.fecha<fecha_salida) then
            leer_venta(vector_detalle[pos],vector_ventas[pos]);
        end;
    end;
procedure actualizar_maximo_minimo(ventas: int16; fecha: t_string20; codigo: int16; var
ventas_max, ventas_min: int16; var fecha_max, fecha_min: t_string20; var codigo_max,
codigo_min: int16);
begin
    if (ventas>ventas_max) then
        begin
            ventas_max:=ventas;
            fecha_max:=fecha;
            codigo_max:=codigo;
        end;
    if (ventas<ventas_min) then
        begin
            ventas_min:=ventas;
            fecha_min:=fecha;
            codigo_min:=codigo;
        end;
    end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_semanario: t_registro_semanario;
    min: t_registro_venta;
    vector_ventas: t_vector_ventas;
    i: t_detalle;
    fecha_max, fecha_min: t_string20;
    ventas, ventas_max, ventas_min, codigo_max, codigo_min: int16;
begin
    ventas_max:=low(int16); fecha_max:=''; codigo_max:=0;
    ventas_min:=high(int16); fecha_min:=''; codigo_min:=0;
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_venta(vector_detalle[i],vector_ventas[i]);
        end;
    minimo(vector_detalle,vector_ventas,min);
    while (min.fecha<>fecha_salida) do
        begin
            read(archivo_maestro,registro_semanario);
            while (registro_semanario.fecha<>min.fecha) do
                read(archivo_maestro,registro_semanario);
            end;
        end;
    end;
end;

```

```

while (registro_semanario.fecha=min.fecha) do
begin
    ventas:=0;
    while (registro_semanario.codigo<>min.codigo) do
        read(archivo_maestro,registro_semanario);
        while ((registro_semanario.fecha=min.fecha) and (registro_semanario.codigo=min.codigo))
do
    begin
        if (registro_semanario.total_ejemplares>=min.ejemplares_vendidos) then
        begin
            registro_semanario.total_ejemplares:=registro_semanario.total_ejemplares-
min.ejemplares_vendidos;
            registro_semanario.ejemplares_vendidos:=registro_semanario.ejemplares_vendidos+min.e
jemplares_vendidos;
            ventas:=ventas+min.ejemplares_vendidos;
        end;
        minimo(vector_detalle,vector_ventas,min);
    end;
    seek(archivo_maestro,filepos(archivo_maestro)-1);
    write(archivo_maestro,registro_semanario);
    actualizar_maximo_minimo(ventas,registro_semanario.fecha,registro_semanario.codigo,venta
s_max,ventas_min,fecha_max,fecha_min,codigo_max,codigo_min);
    end;
end;
close(archivo_maestro);
for i:= 1 to detalles_total do
    close(vector_detalle[i]);
    textcolor(green); write('El semanario que tuvo más ventas fue el '); textcolor(red);
write(codigo_max); textcolor(green); write(' de la fecha '); textcolor(red); write(fecha_max);
textcolor(green); write(', con un total de '); textcolor(red); write(ventas_max);
textcolor(green); writeln(' ventas');
    textcolor(green); write('El semanario que tuvo menos ventas fue el '); textcolor(red);
write(codigo_min); textcolor(green); write(' de la fecha '); textcolor(red); write(fecha_min);
textcolor(green); write(', con un total de '); textcolor(red); write(ventas_min);
textcolor(green); writeln(' ventas');
    writeln();
end;
var
    vector_detalle: t_vector_detalle;
    vector_carga_detalle: t_vector_carga_detalle;
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
    i: t_detalle;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E16_semanariosMaestro');
    assign(archivo_carga_maestro,'E16_semanariosMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ',i,:'); writeln();
            assign(vector_detalle[i],'E16_ventasDetalle'+intToStr(i));
            assign(vector_carga_detalle[i],'E16_ventasDetalle'+intToStr(i)+'.txt');
            cargar_archivo_detalle(vector_detalle[i],vector_carga_detalle[i]);
            imprimir_archivo_detalle(vector_detalle[i]);
        end;
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
    actualizar_archivo_maestro(archivo_maestro,vector_detalle);
    imprimir_archivo_maestro(archivo_maestro);
end.

```

Ejercicio 17.

Una concesionaria de motos de la Ciudad de Chascomús posee un archivo con información de las motos que posee a la venta. De cada moto, se registra: código, nombre, descripción, modelo, marca y stock actual. Mensualmente, se reciben 10 archivos detalles con información de las ventas de cada uno de los 10 empleados que trabajan. De cada archivo detalle, se dispone de la siguiente información: código de moto, precio y fecha de la venta. Se debe realizar un proceso que actualice el stock del archivo maestro desde los archivos detalles. Además, se debe informar cuál fue la moto más vendida.

Nota: Todos los archivos están ordenados por código de la moto y el archivo maestro debe ser recorrido sólo una vez y en forma simultánea con los detalles.

```

program TP2_E17;
{$codepage UTF8}
uses crt, sysutils;
const
    detalles_total=3; // detalles_total=10;
    codigo_salida=999;
type
    t_detalle=1..detalles_total;
    t_string20=string[20];
    t_registro_moto=record
        codigo: int16;
        nombre: t_string20;
        descripcion: t_string20;
        modelo: t_string20;
        marca: t_string20;
        stock: int16;
    end;
    t_registro_venta=record
        codigo: int16;
        precio: real;
        fecha: t_string20;
    end;
    t_archivo_maestro=file of t_registro_moto;
    t_archivo_detalle=file of t_registro_venta;
    t_vector_ventas=array[t_detalle] of t_registro_venta;
    t_vector_detalle=array[t_detalle] of t_archivo_detalle;
    t_vector_carga_detalle=array[t_detalle] of text;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_moto: t_registro_moto;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
        with registro_moto do
            begin
                readln(archivo_carga_maestro,codigo,stock,nombre); nombre:=trim(nombre);
                readln(archivo_carga_maestro,descripcion); descripcion:=trim(descripcion);
                readln(archivo_carga_maestro,modelo); modelo:=trim(modelo);
                readln(archivo_carga_maestro,marca); marca:=trim(marca);
                write(archivo_maestro,registro_moto);
            end;
        end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;

```

```

procedure cargar_archivo_detalle(var archivo_detalle: t_archivo_detalle; var
archivo_carga_detalle: text);
var
    registro_venta: t_registro_venta;
begin
    rewrite(archivo_detalle);
    reset(archivo_carga_detalle);
    while (not eof(archivo_carga_detalle)) do
        with registro_venta do
            begin
                readln(archivo_carga_detalle,codigo,precio,fecha); fecha:=trim(fecha);
                write(archivo_detalle,registro_venta);
            end;
        close(archivo_detalle);
        close(archivo_carga_detalle);
    end;
procedure imprimir_registro_moto(registro_moto: t_registro_moto);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_moto.codigo);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_moto.nombre);
    textcolor(green); write('; Descripción: '); textcolor(yellow);
write(registro_moto.descripcion);
    textcolor(green); write('; Modelo: '); textcolor(yellow); write(registro_moto.modelo);
    textcolor(green); write('; Marca: '); textcolor(yellow); write(registro_moto.marca);
    textcolor(green); write('; Stock: '); textcolor(yellow); writeln(registro_moto.stock);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_moto: t_registro_moto;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro,registro_moto);
            imprimir_registro_moto(registro_moto);
        end;
        close(archivo_maestro);
    end;
procedure imprimir_registro_venta(registro_venta: t_registro_venta);
begin
    textcolor(green); write('Código: '); textcolor(yellow); write(registro_venta.codigo);
    textcolor(green); write('; Precio: $'); textcolor(yellow); write(registro_venta.precio:0:2);
    textcolor(green); write('; Fecha: '); textcolor(yellow); writeln(registro_venta.fecha);
end;
procedure imprimir_archivo_detalle(var archivo_detalle: t_archivo_detalle);
var
    registro_venta: t_registro_venta;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle,registro_venta);
            imprimir_registro_venta(registro_venta);
        end;
        close(archivo_detalle);
    end;
procedure leer_venta(var archivo_detalle: t_archivo_detalle; var registro_venta:
t_registro_venta);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle,registro_venta)
    else
        registro_venta.codigo:=codigo_salida;
    end;
procedure minimo(var vector_detalle: t_vector_detalle; var vector_ventas: t_vector_ventas;
var min: t_registro_venta);

```

```

var
    i, pos: t_detalle;
begin
    min.codigo:=codigo_salida;
    for i:= 1 to detalles_total do
        if (vector_ventas[i].codigo<min.codigo) then
            begin
                min:=vector_ventas[i];
                pos:=i;
            end;
        if (min.codigo<codigo_salida) then
            leer_venta(vector_detalle[pos],vector_ventas[pos]);
    end;
procedure actualizar_maximo(ventas: int16; registro_moto: t_registro_moto; var ventas_max:
int16; var registro_moto_max: t_registro_moto);
begin
    if (ventas>ventas_max) then
        begin
            ventas_max:=ventas;
            registro_moto_max:=registro_moto;
        end;
    end;
procedure actualizar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle: t_vector_detalle);
var
    registro_moto, registro_moto_max: t_registro_moto;
    min: t_registro_venta;
    vector_ventas: t_vector_ventas;
    i: t_detalle;
    ventas, ventas_max: int16;
begin
    ventas_max:=low(int16);
    reset(archivo_maestro);
    for i:= 1 to detalles_total do
        begin
            reset(vector_detalle[i]);
            leer_venta(vector_detalle[i],vector_ventas[i]);
        end;
    minimo(vector_detalle,vector_ventas,min);
    while (min.codigo<>codigo_salida) do
        begin
            ventas:=0;
            read(archivo_maestro,registro_moto);
            while (registro_moto.codigo<>min.codigo) do
                read(archivo_maestro,registro_moto);
            while (registro_moto.codigo=min.codigo) do
                begin
                    ventas:=ventas+1;
                    minimo(vector_detalle,vector_ventas,min);
                end;
            registro_moto.stock:=registro_moto.stock-ventas;
            seek(archivo_maestro,filepos(archivo_maestro)-1);
            write(archivo_maestro,registro_moto);
            actualizar_maximo(ventas,registro_moto,ventas_max,registro_moto_max);
        end;
    close(archivo_maestro);
    for i:= 1 to detalles_total do
        close(vector_detalle[i]);
        textcolor(green); write('La moto más vendida fue la '); textcolor(red);
write(registro_moto_max.marca); textcolor(green); write(' '); textcolor(red);
write(registro_moto_max.modelo); textcolor(green); write(', con un total de ');
textcolor(red); write(ventas_max); textcolor(green); writeln(' ventas');
        writeln();
    end;
var
    vector_detalle: t_vector_detalle;

```

```
vector_carga_detalles: t_vector_carga_detalles;
archivo_maestro: t_archivo_maestro;
archivo_carga_maestro: text;
i: t_detalle;
begin
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
  assign(archivo_maestro, 'E17_motosMaestro');
assign(archivo_carga_maestro, 'E17_motosMaestro.txt');
  cargar_archivo_maestro(archivo_maestro, archivo_carga_maestro);
  imprimir_archivo_maestro(archivo_maestro);
  for i:= 1 to detalles_total do
  begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE ', i, ':'); writeln();
    assign(vector_detalles[i], 'E17_ventasDetalle'+intToStr(i));
assign(vector_carga_detalles[i], 'E17_ventasDetalle'+intToStr(i)+'.txt');
    cargar_archivo_detalle(vector_detalles[i], vector_carga_detalles[i]);
    imprimir_archivo_detalle(vector_detalles[i]);
  end;
  writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO ACTUALIZADO:'); writeln();
  actualizar_archivo_maestro(archivo_maestro, vector_detalles);
  imprimir_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 18.

Se cuenta con un archivo con información de los casos de COVID-19 registrados en los diferentes hospitales de la Provincia de Buenos Aires cada día. Dicho archivo contiene: código de localidad, nombre de localidad, código de municipio, nombre de municipio, código de hospital, nombre de hospital, fecha y cantidad de casos positivos detectados. El archivo está ordenado por localidad, luego por municipio y luego por hospital.

Escribir la definición de las estructuras de datos necesarias y un procedimiento que haga un listado con el siguiente formato:

Nombre: Localidad 1

Nombre: Municipio 1

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio 1

.....
Nombre Municipio N

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio N

Cantidad de casos Localidad 1

Nombre Localidad N

Nombre Municipio 1

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio 1

.....
Nombre Municipio N

Nombre Hospital 1.....Cantidad de casos Hospital 1

.....
Nombre Hospital N.....Cantidad de casos Hospital N

Cantidad de casos Municipio N

Cantidad de casos Localidad N

Cantidad de casos Totales en la Provincia

Además del informe en pantalla anterior, es necesario exportar a un archivo de texto la siguiente información: nombre de localidad, nombre de municipio y cantidad de casos del municipio para aquellos municipios cuya cantidad de casos supere los 1.500. El formato del archivo de texto deberá ser el adecuado para recuperar la información con la menor cantidad de lecturas posibles.

Nota: El archivo debe recorrerse sólo una vez.

```

program TP2_E18;
{$codepage UTF8}
uses crt, sysutils;
const
    codigo_salida=999;
    casos_municipio_corte=1500;
type
    t_string20=string[20];
    t_registro_hospital=record
        codigo_localidad: int16;
        nombre_localidad: t_string20;
        codigo_municipio: int16;
        nombre_municipio: t_string20;
        codigo_hospital: int16;
        nombre_hospital: t_string20;
        fecha: t_string20;
        casos_positivos: int16;
    end;
    t_archivo_maestro=file of t_registro_hospital;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
archivo_carga_maestro: text);
var
    registro_hospital: t_registro_hospital;
begin
    rewrite(archivo_maestro);
    reset(archivo_carga_maestro);
    while (not eof(archivo_carga_maestro)) do
    begin
        with registro_hospital do
        begin
            readln(archivo_carga_maestro,codigo_localidad,codigo_municipio,codigo_hospital,casos_pos
itivos,fecha); fecha:=trim(fecha);
            readln(archivo_carga_maestro,nombre_localidad);
            nombre_localidad:=trim(nombre_localidad);
            readln(archivo_carga_maestro,nombre_municipio);
            nombre_municipio:=trim(nombre_municipio);
            readln(archivo_carga_maestro,nombre_hospital); nombre_hospital:=trim(nombre_hospital);
            write(archivo_maestro,registro_hospital);
        end;
    end;
    close(archivo_maestro);
    close(archivo_carga_maestro);
end;
procedure imprimir_registro_hospital(registro_hospital: t_registro_hospital);
begin
    textcolor(green); write('Código de localidad: '); textcolor(yellow);
write(registro_hospital.codigo_localidad);
    textcolor(green); write('; Nombre de localidad: '); textcolor(yellow);
write(registro_hospital.nombre_localidad);
    textcolor(green); write('; Código de municipio: '); textcolor(yellow);
write(registro_hospital.codigo_municipio);
    textcolor(green); write('; Nombre de municipio: '); textcolor(yellow);
write(registro_hospital.nombre_municipio);
    textcolor(green); write('; Código de hospital: '); textcolor(yellow);
write(registro_hospital.codigo_hospital);
    textcolor(green); write('; Nombre de hospital: '); textcolor(yellow);
write(registro_hospital.nombre_hospital);
    textcolor(green); write('; Fecha: '); textcolor(yellow); write(registro_hospital.fecha);
    textcolor(green); write('; Casos positivos: '); textcolor(yellow);
writeln(registro_hospital.casos_positivos);
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);

```

```

var
    registro_hospital: t_registro_hospital;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
        begin
            read(archivo_maestro, registro_hospital);
            imprimir_registro_hospital(registro_hospital);
        end;
        close(archivo_maestro);
    end;
procedure leer_hospital(var archivo_maestro: t_archivo_maestro; var registro_hospital:
t_registro_hospital);
begin
    if (not eof(archivo_maestro)) then
        read(archivo_maestro, registro_hospital)
    else
        registro_hospital.codigo_localidad:=codigo_salida;
    end;
procedure procesar_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_hospital: t_registro_hospital;
    archivo_txt: text;
    codigo_localidad, codigo_municipio, codigo_hospital, casos_total, casos_localidad,
casos_municipio, casos_hospital: int16;
    nombre_localidad, nombre_municipio, nombre_hospital: t_string20;
begin
    assign(archivo_txt, 'E18_municipiosCasosCorte.txt'); rewrite(archivo_txt);
    casos_total:=0;
    reset(archivo_maestro);
    leer_hospital(archivo_maestro, registro_hospital);
    while (registro_hospital.codigo_localidad<>codigo_salida) do
        begin
            codigo_localidad:=registro_hospital.codigo_localidad;
            nombre_localidad:=registro_hospital.nombre_localidad;
            casos_localidad:=0;
            textcolor(green); write('Localidad: '); textcolor(yellow); writeln(nombre_localidad);
            while (registro_hospital.codigo_localidad=codigo_localidad) do
                begin
                    codigo_municipio:=registro_hospital.codigo_municipio;
                    nombre_municipio:=registro_hospital.nombre_municipio;
                    casos_municipio:=0;
                    textcolor(green); write(' Municipio: '); textcolor(yellow); writeln(nombre_municipio);
                    textcolor(green); writeln(' Hospital Cantidad de casos');
                    while ((registro_hospital.codigo_localidad=codigo_localidad) and
(registro_hospital.codigo_municipio=codigo_municipio)) do
                        begin
                            codigo_hospital:=registro_hospital.codigo_hospital;
                            nombre_hospital:=registro_hospital.nombre_hospital;
                            casos_hospital:=0;
                            while ((registro_hospital.codigo_localidad=codigo_localidad) and
(registro_hospital.codigo_municipio=codigo_municipio) and
(registro_hospital.codigo_hospital=codigo_hospital)) do
                                begin
                                    casos_hospital:=casos_hospital+registro_hospital.casos_positivos;
                                    leer_hospital(archivo_maestro, registro_hospital);
                                end;
                                textcolor(green); write(' '); textcolor(yellow); write(nombre_hospital);
                                textcolor(green); write(' '); textcolor(red); writeln(casos_hospital);
                                casos_municipio:=casos_municipio+casos_hospital;
                            end;
                            textcolor(green); write(' Cantidad de casos Municipio: '); textcolor(red);
writeln(casos_municipio);
                            casos_localidad:=casos_localidad+casos_municipio;
                            if (casos_municipio>casos_municipio_corte) then
                                begin

```

```
        writeln(archivo_txt,casos_municipio,' ',nombre_localidad);
        writeln(archivo_txt,nombre_municipio);
    end;
end;
textcolor(green); write('Cantidad de casos Localidad: '); textcolor(red);
writeln(casos_localidad); writeln();
casos_total:=casos_total+casos_localidad;
end;
textcolor(green); write('Cantidad de casos Totales: '); textcolor(red);
writeln(casos_total);
close(archivo_maestro);
close(archivo_txt);
end;
var
    archivo_maestro: t_archivo_maestro;
    archivo_carga_maestro: text;
begin
    writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
    assign(archivo_maestro,'E18_hospitalesMaestro');
    assign(archivo_carga_maestro,'E18_hospitalesMaestro.txt');
    cargar_archivo_maestro(archivo_maestro,archivo_carga_maestro);
    imprimir_archivo_maestro(archivo_maestro);
    writeln(); textcolor(red); writeln('PROCESAMIENTO ARCHIVO MAESTRO:'); writeln();
    procesar_archivo_maestro(archivo_maestro);
end.
```

Ejercicio 19.

A partir de un siniestro ocurrido, se perdieron las actas de nacimiento y fallecimientos de toda la Provincia de Buenos Aires de los últimos diez años. En pos de recuperar dicha información, se deberá procesar 2 archivos por cada una de las 50 delegaciones distribuidas en la provincia, un archivo de nacimientos y otro de fallecimientos, y crear el archivo maestro reuniendo dicha información.

Los archivos detalles con nacimientos contendrán la siguiente información: nro. partida nacimiento, nombre, apellido, dirección detallada (calle, nro., piso, depto., ciudad), matrícula del médico, nombre y apellido de la madre, DNI madre, nombre y apellido del padre, DNI del padre.

En cambio, los 50 archivos de fallecimientos tendrán: nro. partida nacimiento, DNI, nombre y apellido del fallecido, matrícula del médico que firma el deceso, fecha y hora del deceso y lugar.

Realizar un programa que cree el archivo maestro a partir de toda la información de los archivos detalles. Se debe almacenar en el maestro: nro. partida nacimiento, nombre, apellido, dirección detallada (calle, nro., piso, depto., ciudad), matrícula del médico, nombre y apellido de la madre, DNI madre, nombre y apellido del padre, DNI del padre y, si falleció, además, matrícula del médico que firma el deceso, fecha y hora del deceso y lugar. Se deberá, además, listar, en un archivo de texto, la información recolectada de cada persona.

Nota: Todos los archivos están ordenados por nro. partida de nacimiento, que es única. Tener en cuenta que no necesariamente va a fallecer en el distrito donde nació la persona y, además, puede no haber fallecido.

```
program TP2_E19;
{$codepage UTF8}
uses crt, sysutils;
const
  detalles_nacimientos_total=1; // detalles_nacimientos_total=50;
  detalles_fallecimientos_total=1; // detalles_fallecimientos_total=50;
  nro_partida_salida=999;
type
  t_detalle1=1..detalles_nacimientos_total;
  t_detalle2=1..detalles_fallecimientos_total;
  t_string10=string[10];
  t_registro_direccion=record
    calle: int16;
    nro: int16;
    piso: int16;
    depto: int16;
    ciudad: t_string10;
  end;
  t_registro_persona=record
    nombre: t_string10;
    apellido: t_string10;
    dni: int16;
  end;
  t_registro_deceso=record
    fecha: t_string10;
    hora: t_string10;
    lugar: t_string10;
```

```

end;
t_registro_nacimiento=record
  nro_partida: int16;
  nombre: t_string10;
  apellido: t_string10;
  direccion: t_registro_direccion;
  matricula_medico: int16;
  madre: t_registro_persona;
  padre: t_registro_persona;
end;
t_registro_fallecimiento=record
  nro_partida: int16;
  fallecido: t_registro_persona;
  matricula_medico: int16;
  deceso: t_registro_deceso;
end;
t_registro_acta=record
  nacimiento: t_registro_nacimiento;
  fallecio: boolean;
  matricula_medico: int16;
  deceso: t_registro_deceso;
end;
t_archivo_maestro=file of t_registro_acta;
t_archivo_detalle_nacimientos=file of t_registro_nacimiento;
t_archivo_detalle_fallecimientos=file of t_registro_fallecimiento;
t_vector_nacimientos=array[t_detalle1] of t_registro_nacimiento;
t_vector_fallecimientos=array[t_detalle2] of t_registro_fallecimiento;
t_vector_detalle_nacimientos=array[t_detalle1] of t_archivo_detalle_nacimientos;
t_vector_detalle_fallecimientos=array[t_detalle2] of t_archivo_detalle_fallecimientos;
t_vector_carga_detalle_nacimientos=array[t_detalle1] of text;
t_vector_carga_detalle_fallecimientos=array[t_detalle2] of text;
procedure cargar_archivo_detalle_nacimientos(var archivo_detalle:
t_archivo_detalle_nacimientos; var archivo_carga_detalle: text);
var
  registro_nacimiento: t_registro_nacimiento;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_nacimiento do
      begin
        readln(archivo_carga_detalle,nro_partida,matricula_medico,madre.dni,padre.dni);
        readln(archivo_carga_detalle,nombre); nombre:=trim(nombre);
        readln(archivo_carga_detalle,apellido); apellido:=trim(apellido);
        readln(archivo_carga_detalle,direccion.calle,direccion.nro,direccion.piso,direccion.dept
o,direccion.ciudad); direccion.ciudad:=trim(direccion.ciudad);
        readln(archivo_carga_detalle,madre.nombre); madre.nombre:=trim(madre.nombre);
        readln(archivo_carga_detalle,madre.apellido); madre.apellido:=trim(madre.apellido);
        readln(archivo_carga_detalle,padre.nombre); padre.nombre:=trim(padre.nombre);
        readln(archivo_carga_detalle,padre.apellido); padre.apellido:=trim(padre.apellido);
        write(archivo_detalle,registro_nacimiento);
      end;
    end;
  close(archivo_detalle);
  close(archivo_carga_detalle);
end;
procedure cargar_archivo_detalle_fallecimientos(var archivo_detalle:
t_archivo_detalle_fallecimientos; var archivo_carga_detalle: text);
var
  registro_fallecimiento: t_registro_fallecimiento;
begin
  rewrite(archivo_detalle);
  reset(archivo_carga_detalle);
  while (not eof(archivo_carga_detalle)) do
    with registro_fallecimiento do
      begin
        readln(archivo_carga_detalle,nro_partida,matricula_medico,fallecido.dni);

```

```

        readln(archivo_carga_detalle,fallecido.nombre);
fallecido.nombre:=trim(fallecido.nombre);
        readln(archivo_carga_detalle,fallecido.apellido);
fallecido.apellido:=trim(fallecido.apellido);
        readln(archivo_carga_detalle,deceso.fecha); deceso.fecha:=trim(deceso.fecha);
        readln(archivo_carga_detalle,deceso.hora); deceso.hora:=trim(deceso.hora);
        readln(archivo_carga_detalle,deceso.lugar); deceso.lugar:=trim(deceso.lugar);
        write(archivo_detalle,registro_fallecimiento);
    end;
    close(archivo_detalle);
    close(archivo_carga_detalle);
end;
procedure imprimir_registro_direccion(registro_direccion: t_registro_direccion);
begin
    textcolor(green); write('; Calle: '); textcolor(yellow); write(registro_direccion.calle);
    textcolor(green); write('; Nro.: '); textcolor(yellow); write(registro_direccion.nro);
    textcolor(green); write('; Piso: '); textcolor(yellow); write(registro_direccion.piso);
    textcolor(green); write('; Depto.: '); textcolor(yellow); write(registro_direccion.depto);
    textcolor(green); write('; Ciudad: '); textcolor(yellow); write(registro_direccion.ciudad);
end;
procedure imprimir_registro_persona(registro_persona: t_registro_persona; persona: string);
begin
    textcolor(green); write('; Nombre ',persona,': '); textcolor(yellow);
write(registro_persona.nombre);
    textcolor(green); write('; Apellido ',persona,': '); textcolor(yellow);
write(registro_persona.apellido);
    if (persona<>'padre') then
    begin
        textcolor(green); write('; DNI ',persona,': '); textcolor(yellow);
write(registro_persona.dni);
    end
    else
    begin
        textcolor(green); write('; DNI ',persona,': '); textcolor(yellow);
writeln(registro_persona.dni);
    end;
end;
procedure imprimir_registro_nacimiento(registro_nacimiento: t_registro_nacimiento);
begin
    textcolor(green); write('Nro. partida nacimiento: '); textcolor(yellow);
write(registro_nacimiento.nro_partida);
    textcolor(green); write('; Nombre: '); textcolor(yellow); write(registro_nacimiento.nombre);
    textcolor(green); write('; Apellido: '); textcolor(yellow);
write(registro_nacimiento.apellido);
    imprimir_registro_direccion(registro_nacimiento.direccion);
    textcolor(green); write('; Matrícula médico: '); textcolor(yellow);
write(registro_nacimiento.matricula_medico);
    imprimir_registro_persona(registro_nacimiento.madre,'madre');
    imprimir_registro_persona(registro_nacimiento.padre,'padre');
end;
procedure imprimir_archivo_detalle_nacimientos(var archivo_detalle:
t_archivo_detalle_nacimientos);
var
    registro_nacimiento: t_registro_nacimiento;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
    begin
        read(archivo_detalle,registro_nacimiento);
        imprimir_registro_nacimiento(registro_nacimiento);
    end;
    close(archivo_detalle);
end;
procedure imprimir_registro_deceso(registro_deceso: t_registro_deceso);
begin

```

```

    textcolor(green); write('; Fecha deceso: '); textcolor(yellow);
write(registro_deceso.fecha);
    textcolor(green); write('; Hora deceso: '); textcolor(yellow); write(registro_deceso.hora);
    textcolor(green); write('; Lugar deceso: '); textcolor(yellow);
writeln(registro_deceso.lugar);
end;
procedure imprimir_registro_fallecimiento(registro_fallecimiento: t_registro_fallecimiento);
begin
    textcolor(green); write('Nro. partida nacimiento: '); textcolor(yellow);
write(registro_fallecimiento.nro_partida);
    imprimir_registro_persona(registro_fallecimiento.fallecido, 'fallecido');
    textcolor(green); write('; Matrícula médico deceso: '); textcolor(yellow);
write(registro_fallecimiento.matricula_medico);
    imprimir_registro_deceso(registro_fallecimiento.deceso);
end;
procedure imprimir_archivo_detalle_fallecimientos(var archivo_detalle:
t_archivo_detalle_fallecimientos);
var
    registro_fallecimiento: t_registro_fallecimiento;
begin
    reset(archivo_detalle);
    while (not eof(archivo_detalle)) do
        begin
            read(archivo_detalle, registro_fallecimiento);
            imprimir_registro_fallecimiento(registro_fallecimiento);
        end;
        close(archivo_detalle);
    end;
procedure leer_nacimiento(var archivo_detalle: t_archivo_detalle_nacimientos; var
registro_nacimiento: t_registro_nacimiento);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_nacimiento)
    else
        registro_nacimiento.nro_partida:=nro_partida_salida;
    end;
procedure leer_fallecimiento(var archivo_detalle: t_archivo_detalle_fallecimientos; var
registro_fallecimiento: t_registro_fallecimiento);
begin
    if (not eof(archivo_detalle)) then
        read(archivo_detalle, registro_fallecimiento)
    else
        registro_fallecimiento.nro_partida:=nro_partida_salida;
    end;
procedure minimo_nacimiento(var vector_detalle_nacimientos: t_vector_detalle_nacimientos;
var vector_nacimientos: t_vector_nacimientos; var min_nacimiento: t_registro_nacimiento);
var
    i, pos: t_detalle1;
begin
    min_nacimiento.nro_partida:=nro_partida_salida;
    for i:= 1 to detalles_nacimientos_total do
        if (vector_nacimientos[i].nro_partida<min_nacimiento.nro_partida) then
            begin
                min_nacimiento:=vector_nacimientos[i];
                pos:=i;
            end;
        if (min_nacimiento.nro_partida<nro_partida_salida) then
            leer_nacimiento(vector_detalle_nacimientos[pos], vector_nacimientos[pos]);
        end;
procedure minimo_fallecimiento(var vector_detalle_fallecimientos:
t_vector_detalle_fallecimientos; var vector_fallecimientos: t_vector_fallecimientos; var
min_fallecimiento: t_registro_fallecimiento);
var
    i, pos: t_detalle2;
begin
    min_fallecimiento.nro_partida:=nro_partida_salida;

```

```

for i:= 1 to detalles_fallecimientos_total do
  if (vector_fallecimientos[i].nro_partida<min_fallecimiento.nro_partida) then
    begin
      min_fallecimiento:=vector_fallecimientos[i];
      pos:=i;
    end;
  if (min_fallecimiento.nro_partida<nro_partida_salida) then
    leer_fallecimiento(vector_detalle_fallecimientos[pos],vector_fallecimientos[pos]);
end;
procedure exportar_archivo_txt(var archivo_maestro: t_archivo_maestro);
var
  registro_acta: t_registro_acta;
  archivo_txt: text;
  i: int16;
begin
  i:=1;
  reset(archivo_maestro);
  assign(archivo_txt,'E19_actasMaestro.txt'); rewrite(archivo_txt);
  while (not eof(archivo_maestro)) do
    begin
      read(archivo_maestro,registro_acta);
      with registro_acta do
        begin
          writeln(archivo_txt,'ACTA ',i);
          writeln(archivo_txt,'Nro. partida nacimiento: ',nacimient.nro_partida);
          writeln(archivo_txt,'Nombre: ',nacimient.nombre);
          writeln(archivo_txt,'Apellido: ',nacimient.apellido);
          writeln(archivo_txt,'Dirección: Calle ',nacimient.direccion.calle,', Nro.
',nacimient.direccion.nro,', Piso ',nacimient.direccion.piso,', Depto.
',nacimient.direccion.depto,', Ciudad ',nacimient.direccion.ciudad);
          writeln(archivo_txt,'Matrícula médico: ',nacimient.matricula_medico);
          writeln(archivo_txt,'Madre: Nombre ',nacimient.madre.nombre,', Apellido
',nacimient.madre.apellido,', DNI ',nacimient.madre.dni);
          writeln(archivo_txt,'Padre: Nombre ',nacimient.padre.nombre,', Apellido
',nacimient.padre.apellido,', DNI ',nacimient.padre.dni);
          if (fallecio=true) then
            begin
              writeln(archivo_txt,'Falleció: Sí');
              writeln(archivo_txt,'Matrícula médico deceso: ',matricula_medico);
              writeln(archivo_txt,'Deceso: Fecha ',deceso.fecha,', Hora ',deceso.hora,', Lugar
',deceso.lugar);
            end
          else
            begin
              writeln(archivo_txt,'Falleció: No');
            end;
          writeln(archivo_txt);
        end;
      i:=i+1;
    end;
  close(archivo_txt);
end;
procedure cargar_archivo_maestro(var archivo_maestro: t_archivo_maestro; var
vector_detalle_nacimientos: t_vector_detalle_nacimientos; var
vector_detalle_fallecimientos: t_vector_detalle_fallecimientos);
var
  registro_acta: t_registro_acta;
  min_nacimiento: t_registro_nacimiento;
  min_fallecimiento: t_registro_fallecimiento;
  vector_nacimientos: t_vector_nacimientos;
  vector_fallecimientos: t_vector_fallecimientos;
  i: t_detalle1;
  j: t_detalle2;
begin
  rewrite(archivo_maestro);
  for i:= 1 to detalles_nacimientos_total do

```

```

begin
    reset(vector_detalle_nacimientos[i]);
    leer_nacimiento(vector_detalle_nacimientos[i],vector_nacimientos[i]);
end;
for j:= 1 to detalles_fallecimientos_total do
begin
    reset(vector_detalle_fallecimientos[j]);
    leer_fallecimiento(vector_detalle_fallecimientos[j],vector_fallecimientos[j]);
end;
minimo_nacimiento(vector_detalle_nacimientos,vector_nacimientos,min_nacimiento);
minimo_fallecimiento(vector_detalle_fallecimientos,vector_fallecimientos,min_fallecimiento)
;
while (min_nacimiento.nro_partida<>nro_partida_salida) do
begin
    registro_acta.nacimiento:=min_nacimiento;
    if (min_nacimiento.nro_partida=min_fallecimiento.nro_partida) then
begin
    registro_acta.fallecio:=true;
    registro_acta.matricula_medico:=min_fallecimiento.matricula_medico;
    registro_acta.deceso:=min_fallecimiento.deceso;
    minimo_fallecimiento(vector_detalle_fallecimientos,vector_fallecimientos,min_fallecimie
nto);
end
else
    registro_acta.fallecio:=false;
    write(archivo_maestro,registro_acta);
    minimo_nacimiento(vector_detalle_nacimientos,vector_nacimientos,min_nacimiento);
end;
exportar_archivo_txt(archivo_maestro);
close(archivo_maestro);
for i:= 1 to detalles_nacimientos_total do
    close(vector_detalle_nacimientos[i]);
for j:= 1 to detalles_fallecimientos_total do
    close(vector_detalle_fallecimientos[j]);
end;
procedure imprimir_registro_acta(registro_acta: t_registro_acta);
begin
    imprimir_registro_nacimiento(registro_acta.nacimiento);
    if (registro_acta.fallecio=true) then
begin
        textcolor(green); write('; Falleció: '); textcolor(yellow); write('Sí');
        textcolor(green); write('; Matrícula médico deceso: '); textcolor(yellow);
write(registro_acta.matricula_medico);
        imprimir_registro_deceso(registro_acta.deceso);
end
else
begin
        textcolor(green); write('; Falleció: '); textcolor(yellow); writeln('No');
end;
end;
procedure imprimir_archivo_maestro(var archivo_maestro: t_archivo_maestro);
var
    registro_acta: t_registro_acta;
begin
    reset(archivo_maestro);
    while (not eof(archivo_maestro)) do
begin
        read(archivo_maestro,registro_acta);
        imprimir_registro_acta(registro_acta);
end;
    close(archivo_maestro);
end;
var
    vector_detalle_nacimientos: t_vector_detalle_nacimientos;
    vector_detalle_fallecimientos: t_vector_detalle_fallecimientos;
    vector_carga_detalle_nacimientos: t_vector_carga_detalle_nacimientos;

```

```
vector_carga_detalle_fallecimientos: t_vector_carga_detalle_fallecimientos;
archivo_maestro: t_archivo_maestro;
i: t_detalle1;
j: t_detalle2;
begin
  for i:= 1 to detalles_nacimientos_total do
    begin
      writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE NACIMIENTOS ',i,':');
      writeln();
      assign(vector_detalle_nacimientos[i], 'E19_nacimientosDetalle'+intToStr(i));
      assign(vector_carga_detalle_nacimientos[i], 'E19_nacimientosDetalle'+intToStr(i)+'.txt');
      cargar_archivo_detalle_nacimientos(vector_detalle_nacimientos[i], vector_carga_detalle_nacimientos[i]);
      imprimir_archivo_detalle_nacimientos(vector_detalle_nacimientos[i]);
    end;
    for j:= 1 to detalles_fallecimientos_total do
      begin
        writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO DETALLE FALLECIMIENTOS ',j,':');
        writeln();
        assign(vector_detalle_fallecimientos[j], 'E19_fallecimientosDetalle'+intToStr(j));
        assign(vector_carga_detalle_fallecimientos[j], 'E19_fallecimientosDetalle'+intToStr(j)+'.txt');
        cargar_archivo_detalle_fallecimientos(vector_detalle_fallecimientos[j], vector_carga_detalle_fallecimientos[j]);
        imprimir_archivo_detalle_fallecimientos(vector_detalle_fallecimientos[j]);
      end;
      writeln(); textcolor(red); writeln('IMPRESIÓN ARCHIVO MAESTRO:'); writeln();
      assign(archivo_maestro, 'E19_actasMaestro');
      cargar_archivo_maestro(archivo_maestro, vector_detalle_nacimientos, vector_detalle_fallecimientos);
      imprimir_archivo_maestro(archivo_maestro);
    end.
```